An Empirical-Based Security Benchmarking Framework for Container Confinement

by

Huzaifa Patel

A thesis submitted to the Faculty of Graduate and Postdoctoral Affairs
in partial fulfillment of the requirements for the degree of

Master
in
Computer Science

Carleton University
Ottawa, Ontario

© 2025 Huzaifa Patel

Contents

1 Introduction **7**
- 1.1 Contributions . . . 9
- 1.2 The Container Confinement Problem . . . 10
- 1.3 Motivation . . . 11
  - 1.3.1 Example Applications of Houdini . . . 12
- 1.4 Related Publication . . . 14

2 Background & Related Work **15**
- 2.1 Type 1 Hypervisor . . . 16
- 2.2 Type 2 Hypervisor . . . 16
- 2.3 Problems With Type 1 & Type 2 Hypervisor Classifications . . . 16
- 2.4 Native Hypervisor . . . 17
- 2.5 Emulation Hypervisor . . . 17
- 2.6 Why do we have run Docker with sudo? . . . 17
- 2.7 cgroups . . . 18
- 2.8 namespaces . . . 18
- 2.9 Linux capabilties . . . 19
- 2.10 Seccomp (Secure Computing Mode) . . . 20
- 2.11 App Armor (Application Armor) . . . 20
- 2.12 selinux . . . 21
- 2.13 YAMA . . . 21
- 2.14 Kernel Implementation . . . 24
- 2.15 Enforcement of `ptrace_scope` . . . 25
- 2.16 `ptrace_scope` Settings . . . 25
- 2.17 Related Work . . . 25
  - 2.17.1 Apparmor . . . 30
  - 2.17.2 Control Groups . . . 31
  - 2.17.3 Extended Berkeley Packet Filter . . . 33
  - 2.17.4 System Call Filtering . . . 36
  - 2.17.5 Rule-based Systems . . . 39
  - 2.17.6 OS Changes . . . 43
  - 2.17.7 Secure Computing . . . 45

3 Design **47**
3.1 System Architecture . . . 48
3.2 Reproducibility . . . 50
3.3 Scalability . . . 52
3.4 Host Isolation . . . 53
3.5 Transparency and Debuggability . . . 54
3.5.1 Other Design Goals . . . 55

4 Implementation **57**
4.1 Trick Structure Overview . . . 57
4.2 YAML-Based Trick Definition . . . 57
4.3 Configuration File . . . 58
4.4 Trick Script . . . 60
4.5 Dockerfile . . . 62
4.6 Python-Based Parameter Expansion Mode . . . 64
4.7 Execution Pipeline . . . 67
4.8 Example Trick: Shadow File Access . . . 68
4.9 Flask Server Integration . . . 69
4.10 Trick Listener and Trick Execution Logic . . . 72
4.11 Version Reporting Endpoint . . . 75
4.12 Dynamic Runtime Dispatch . . . 77
4.13 Trick Parameter Expansion Logic . . . 79
4.14 Security Considerations in Host-Level Code . . . 81
4.15 Virtual Machine Setup and Execution . . . 82
4.15.1 Purpose of Virtualization . . . 82
4.15.2 QEMU Execution Command . . . 83
4.15.3 Key VM Configuration Options . . . 83
4.15.4 Lifecycle of a Virtual Machine . . . 84

5 Evaluation **86**
5.1 Reproducibility Across Time with Version Pinning . . . 87
5.2 Reproducibility Across Machines . . . 92
5.3 Host Isolation Validation . . . 95
5.3.1 Threat Model . . . 98
5.3.2 Baseline Comparison: Without VM Isolation . . . 99
5.3.3 Consideration of VM Escape Attacks . . . 100
5.4 Extensibility Evaluation . . . 101
5.5 Version Variability Support . . . 104
5.6 Performance Benchmarking . . . 109
5.6.1 Understanding Time Metrics in Microbenchmarking . . . 109
5.6.2 Microbenchmark . . . 110
5.7 Batch Trick Execution . . . 114
5.8 Scalability via Bulk Execution . . . 114

6 Case Studies **117**
6.1 Evaluating the accuracy of CVE-2024-21626 117
6.2 Evaluating Host Process Manipulation 119
6.3 Evaluating Docker Privileged Port Forwarding Restrictions 123
6.4 Evaluating ptrace Enforcement Across Linux Distributions 126

7 Discussion **128**

List of Tables

2.1 Yama `ptrace_scope` Settings 23

5.1 Virtual Machine Experimental Configuration 88
5.2 Version-Pinned Components 88
5.3 Reproducibility Verification Across Time 88
5.4 Comparison of Machine A and Machine B 92
5.5 Reproducibility Comparison Across Machines 94
5.6 Docker and runc Versions Validated with Houdini 105
5.7 Performance Metrics for `mkfs` Stage 110
5.8 Performance Metrics for VM Boot 111
5.9 Performance Metrics for Docker Image Build (Cold Start) 112
5.10 Your Caption 113
5.11 Performance Metrics for Manually Executed Baseline 114
5.12 Performance Metrics for Batch Execution of 32 Tricks 115

6.1 runc versions against CVE-2024-21626 119
6.2 Case Study 2 Testing Environment 120
6.3 Ability for Containers to Interact with Host Processes Across Yama Settings (Ubuntu Kernel 6.8.12) 123
6.4 * 123
6.5 Ability for a container to bind to privileged ports 124
6.6 * 124
6.7 ptrace Behavior Across Distributions with Default Yama Scopes . . . 127

List of Figures

3.1 Architecture of Houdini 50

4.1 Configuration file for a default Docker out-of-the-box security setting. 60
4.2 Example trick implementation that checks for read access to `/etc/shadow`. 62
4.3 Example Dockerfile used to prepare the container environment for a Python-based trick. 64

Nomenclature

VM virtual machine. 15

Chapter 1

Introduction

Container-based workloads have become a cornerstone of modern cloud infrastructure, revolutionizing how applications are developed, deployed, and scaled, particularly within microservices architectures and DevOps workflows. Unlike hypervisor-based virtual machines (VMs), which each require a full operating system and rely on a hypervisor for hardware virtualization, containers are lightweight, share the host system's kernel, and incur minimal overhead. This efficiency makes them exceptionally well-suited for continuous integration and rapid scaling, allowing organizations to optimize resource utilization and reduce costs while ensuring quick, consistent deployments. Furthermore, containers enable seamless portability by packaging all userspace dependencies into a unified image, thereby avoiding the common problem of environment-specific discrepancies. Yet, despite containers' numerous benefits, they also introduce new security challenges, especially where container confinement is concerned.

Linux kernel features such as cgroups and namespaces provide resource, process, and network isolation for containers. However, this separation is not foolproof. Kernel vulnerabilities, such as, but not limited to buffer overflows or privilege escalation flaws can allow a process to circumvent the very mechanisms designed to confine it. Mis-

configurations can similarly open holes in container security, for instance by granting unnecessary capabilities, mounting sensitive host directories, or running containers in privileged mode. Container escapes exploit these weaknesses to break out of the container's resource and namespace isolation. Once an attacker gains elevated privileges or direct access to the host system, they can disrupt other containers, exfiltrate data, or compromise the entire infrastructure.

Technologies like SELinux, AppArmor, and seccomp provide additional layers of host-based security to mitigate these issues but are notoriously prone to misconfiguration. Best practices guides for container security [**?**] can help but they offer no guarantee that a given configuration will perform as intended by the end user. As container adoption has skyrocketed across industries, particularly in multi-tenant cloud environments, these security challenges have become more pronounced.

To improve container security, the cloud industry has developed technologies like Kata Containers[1], which isolate containers further by running them within lightweight VMs. While this approach can be effective, it also adds another layer of complexity which is subject to its own set of vulnerabilities and implementation errors. Simply adding additional layers of security does not, it itself, mean that security goals have been achieved.

Test suites play a crucial role in verifying that systems meet specified requirements and function correctly under defined conditions. They are widely used across computing disciplines, from performance benchmarking to security assessment. For instance, the Standard Performance Evaluation Corporation (SPEC) [**?**] is best known for developing industry-standard benchmarks that evaluate CPU performance and power efficiency, while the Transaction Processing Performance Council (TPC) [**?**] establishes benchmarking standards for database performance. Specialized security-focused test suites play a vital role in detecting vulnerabilities in deployed systems.

[1] `https://katacontainers.io`

Tools such as network scanners aid in identifying misconfigurations, outdated software versions, and unintended service exposures, helping organizations proactively mitigate security risks. However, a significant gap remains in the context of containers. There are currently no standardized or widely adopted test suites designed specifically to evaluate container confinement, functionality, or isolation effectiveness. This absence leaves organizations to rely on ad hoc methods, which are often inadequate for uncovering subtle misconfigurations or ensuring robust confinement. Given the growing reliance on containerized environments in modern infrastructure, the development of comprehensive test suites tailored for containers is imperative.

In this paper, we describe Houdini, the first test suite for verifying container confinement. Given a Docker container configuration (including the kernel, docker, and runc version), Houdini will instantiate the container in a standalone QEMU-based VM and perform multiple tests (or *tricks*, as we call them) to evaluate the container's security posture. These tests explore vulnerabilities such as improper resource allocation, inadequate isolation, and insecure filesystem setups. Houdini is written in Python and provides an extension language for writing tricks, allowing it to adapt as new container escape vulnerabilities arise.

1.1 Contributions

This paper makes the following contributions:

- Defines container confinement and its associated challenges.
- Introduces Houdini, a modular framework for testing container security.
- Demonstrates Houdini's ability to detect security issues, misconfigurations, and inconsistencies in container confinement.

1.2 The Container Confinement Problem

Container confinement can be defined as the set of mechanisms and strategies used to isolate a containerized application from the host system and other containers. Effective confinement is critical for maintaining system security and stability, as it ensures that containers operate within defined boundaries by restricting their access to system resources including processes, networks, filesystems, devices, and kernel interfaces.

A challenge with container confinement is that the level of confinement of a container is not fixed; it can vary based on the specific requirements of the user. In other words, the degree to which a container is isolated from the host system or other containers is customizable to meet the user's needs. At one end of the spectrum, one can lock a container down so tightly that it is almost entirely isolated from the host system, which minimizes potential attack surfaces but can also render the container incapable of performing key tasks. On the other hand, giving a container too many privileges increases the impact of any security compromise. The optimal configuration is one that strikes a balance between maximum confinement (i.e., least privilege) and sufficient functionality.

A standardized, formal approach involving rigorously evaluating and analyzing the security of the container and its environment might seem ideal; however, it is impractical because container environments are highly complex, with interactions between kernels, runtimes, orchestration tools, and external dependencies that are difficult to model comprehensively. Moreover, many vulnerabilities stem from subtle details that a formal model might overlook. Instead, we believe an empirical approach offers a practical alternative. Although it will likely never be exhaustive, the goal is not comprehensive coverage but rather establishing a baseline set of tests that demonstrate effective container confinement.

1.3 Motivation

Houdini is designed to provide a systematic and reproducible approach to testing container confinement mechanisms. One of its primary goals is to evaluate whether container security features, such as namespaces, cgroups, seccomp, and Linux capabilities are effectively enforcing isolation. Many container security tools rely on theoretical guarantees or static analysis[**?**], but Houdini focuses on empirical validation by executing controlled security tests that verify the correct enforcement of security mechanisms. By doing so, Houdini ensures that container security mechanisms are correctly configured.

Houdini is not only for testing container isolation but also as a means to optimize container configurations. By systematically simulating various tasks within different containers, Houdini can help determine the minimal set of privileges required for a container to perform its intended function. For example, administrators can start with a container configured with broad privileges and then incrementally reduce permissions, such as Linux capabilities, seccomp filters, or resource limits, while monitoring task success. Houdini's testing framework will identify the point at which functionality is maintained while unnecessary privileges are removed, thereby pinpointing the least privilege configuration that still allows the container to operate effectively. Minimizing the privileges granted to a container reduces its attack surface, making it more difficult for an attacker to exploit vulnerabilities.

Although runtime monitoring and anomaly detection, such as with eBPF-based tracing[1] can provide immediate insights into container behavior, our goal isn't to implement an exhaustive, continuous surveillance system. Instead, we aim to focus on auditing and validating the underlying isolation mechanisms to ensure they are correctly configured and consistently enforced.

1.3.1 Example Applications of Houdini

A system like Houdini can be used in the following ways:

Identify gaps between intended confinement policies and actual enforcement: There is often a discrepancy between the security policies that administrators intend to enforce and what is actually upheld at runtime. These discrepancies generally fall into two categories: implicit policy problems and conflicting policy configurations.

An implicit policy problem happens when an administrator doesn't explicitly set a security rule, and the system's default behavior is used instead. These defaults might lead to behavior the administrator did not expect. For example, Docker containers are, by default, connected to a shared internal bridge network. While this topology typically prevents internet access, it still allows containers to communicate with each other. An administrator might see that outbound traffic from a container is blocked and mistakenly believe the container is fully isolated from other networks. However, internal communication between containers will still be allowed unless it's explicitly disabled. Thus, this default behavior can create a false sense of security and introduce unexpected risks.

A conflicting policy configuration, on the other hand, occurs when a security policy is correctly specified but is later invalidated by other conflicting configurations. For example, an administrator might launch a container with `-network=none`, expecting it to be fully isolated from all network communication. However, if the container is also inadvertently given elevated privileges such as `-cap-add=NET_ADMIN` or `-privileged`, it can reconfigure its own network interfaces and regain connectivity. With the `NET_ADMIN` capability, processes inside the container gain access to low-level networking operations, such as creating virtual interfaces, assigning IP addresses, or modifying routing tables. A process could use commands like `ip link set eth0 up` and `dhclient eth0` to bring up a previously disabled interface and request an IP

address from the host's network. In the case of a `-privileged` container, even more capabilities are granted, allowing near-complete access to the host's kernel features, including the creation of new network namespaces or interfaces, effectively bypassing the isolation intended by `-network=none`.

Houdini can help identify these gaps by systematically testing the behavior of the container against the intended security policies, as demonstrated in case study 2 in .

Challenge misleading security assumptions with empirical testing: Container security is often discussed in terms of theoretical guarantees, with many security best practices based on assumptions rather than direct testing. For instance, documentation may state that a container running in unprivileged mode cannot escalate to root, but Houdini actively tests whether that is actually the case under different container configurations. By running real-world security validation instead of relying solely on security claims, Houdini helps challenge misleading assumptions and exposes where security mechanisms fail in practice. This is particularly important as container escapes and privilege escalation exploits continue to emerge, demonstrating that security controls do not always work as expected. By grounding security claims in measurable results rather than theory, Houdini helps researchers and practitioners develop more robust container security strategies.

Detect security drift over time: One of the challenges in maintaining secure containerized environments is configuration drift. Security settings applied during container initialization may be inadvertently changed through image updates, runtime automation, or infrastructure changes. Houdini can be set up to periodically re-evaluate security policies, making it possible to detect when the system diverges from its originally intended security posture. This allows for the timely remediation of regressions that could otherwise go unnoticed. Configuration drift does not only arise from explicit policy changes, but also from underlying updates to system components such as the Linux kernel, Docker, or runc. Even when container configurations and

security policies remain unchanged, an upgrade to the kernel or container runtime can subtly alter the enforcement of isolation primitives, security defaults, or available capabilities. For example, a new kernel version may modify the behavior of namespaces, cgroups, or seccomp filters; a Docker update may introduce new features or alter default permissions.

1.4 Related Publication

The majority of the content presented in this thesis has been published as a peer-reviewed publication and accepted for publication, respectively. Both publications were written primarily by the author of this thesis, Huzaifa Patel under the supervision of his supervisor Dr. David Barrera, and Dr. Anil Somayaji who both played a crucial role in supervising the publications.

Chapter 2

Background & Related Work

The existance of cloud computing can be attributed to virtualization, which is a technology that makes it possible for multiple different operating systems (OSs) to run concurrently, and in an isolated environment on the same hardware. How does virtualization achieve this? It makes use of a machines hardware to support the software that creates and manages virtual machines (VMs). A virtual machine (VM) is a virtual environment that provides the functionality of a physical computer by using its own virtual central processing unit (VCPU), memory, network interface, and storage. The software that creates and manages VMs is formally called a hypervisor or virtual machine monitor (VMM). The virtualization marketplace is comprised of four notable hypervisors, which are: (1) VMWare, (2) Xen, (3) Kernel-based Virtual Machine (KVM), and (4) Hyper-V. The operating system running a hypervisor is called the host OS, while the VM that uses the hypervisor is called the guest OS. To maintain consistency within this proposal, we will use the word "hypervisor" when referring to virtualization software. Hypervisors can be divided into two types, depending on where they are located on the machine: (1) type 1 and (2) type 2.

2.1 Type 1 Hypervisor

Type 1 hypervisors run directly on physical hardware to create, control, and manage VMs, and do not require support from the host OS. Type 1 hypervisors are also called native or bare-metal hypervisors. The first hypervisors, which IBM developed in the 1960s were type 1 hypervisors [?]. Examples of type 1 hypervisors include, but are not limited to Xen, KVM, VMware ESX, and Microsoft Hyper-V [?].

2.2 Type 2 Hypervisor

Type 2 hypervisors (also called hosted hypervisors) are installed on the OS (Windows, Linux, MacOS), similar to how computer programs are installed onto an OS. In other words, a type 2 hypervisor runs as a process on the host OS. Type 2 hypervisors abstract guest OSs from the host OS by introducing a third software layer above the hardware, as shown in figure 2.1. Examples of type 2 hypervisors include but are not limited to VMware Workstation, VirtualBox, and QEMU [?].

2.3 Problems With Type 1 & Type 2 Hypervisor Classifications

Although the definitions of type 1 and type 2 hypervisors are widely accepted, there are gray areas where the distinction between the two remain unclear. For instance, according to its creator RedHat, KVM is implemented and deployed using two Linux kernel modules that effectively convert the host Linux OS into a type-1 hypervisor [?]. At the same time, KVM can be categorized as a type 2 hypervisor because KVM VM's are standard Linux processes that are competing with other Linux processes for CPU time given by the Linux kernel's native CPU scheduler [?]. Due to disagreements and vagueness in the definitions of type 1 and type 2 hypervisors, two new classifications

were defined: (1) native hypervisors and (2) emulation hypervisors [?].

2.4 Native Hypervisor

Native hypervisors are hypervisors that push VM guest instructions directly to the physical machines CPU, by using virtualization extensions like Intel VT-x. [?]. Examples of Native hypervisors include but are not limited to Xen, KVM, VMware ESX, and Microsoft HyperV.

2.5 Emulation Hypervisor

Emulation hypervisors are hypervisors that emulate every VM guest instruction using software virtualization [?]. Emulated guest instructions are very easy to trace because all guest VM instructions are trapped to the hypervisor. Examples of emulation hypervisors include but are not limited to QEMU, Bochs, early versions of VMware-Workstation, and VirtualBox [?].

2.6 Why do we have run Docker with sudo?

By default, Docker's daemon (dockerd) runs as root, and access to it is controlled through the Unix socket:

Copy Edit /var/run/docker.sock This socket is owned by root and typically part of the docker group. If your user isn't in the docker group, you'll need sudo to interact with it.

Why Docker Needs Elevated Privileges: Container Isolation Requires Kernel Features:

Managing cgroups, namespaces, mounting filesystems, setting up virtual networks—all require elevated permissions.

By Default, Docker Protects Access to Its Control API:

docker.sock is a privileged control interface that allows full system control (you can effectively become root via it).

On Linux, process containment relies on several key technologies, including Unix, cgroups, namespaces, Linux capabilities, apparmor, selinux, and seccomp. Below, we provide an overview of each of these mechanisms.

2.7 cgroups

Cgroups, or control groups, are a Linux kernel feature that allows system administrators to allocate and manage system resources (such as CPU, memory, disk I/O, and network bandwidth) for processes or groups of processes. When you create a cgroup, you are essentially grouping together processes and imposing resource constraints on them. This is crucial for resource management and isolation in multi-user or containerized environments. For instance, cgroups allow you to set a limit on the amount of memory a set of processes can use, preventing any single process from consuming excessive resources and affecting other processes or the system as a whole. Additionally, cgroups allow for the prioritization of resources, ensuring that critical processes receive more CPU or memory than less important ones. They also offer monitoring capabilities to track how much of each resource a process or cgroup is consuming. This feature is extensively used in containerization technologies like Docker, where containers are assigned their own resource limits via cgroups to ensure fairness and prevent resource hogging.

2.8 namespaces

Namespaces are another crucial Linux kernel feature that provides process isolation. A namespace isolates certain aspects of a system for a process or group of processes,

ensuring that they have their own independent environment. There are several types of namespaces, including PID, network, mount, user, and UTS namespaces. For example, a PID namespace allows processes within it to have their own process IDs, meaning that processes in different PID namespaces can have the same process ID without interfering with each other. Similarly, network namespaces allow processes to have their own network stack, which includes IP addresses, routing tables, and network interfaces. This isolation is fundamental for containers, as it ensures that processes running in one container cannot see or interact with processes in another, even though they may be running on the same physical host. Namespaces, combined with cgroups, provide the foundation for building lightweight, secure containers in Linux systems.

2.9 Linux capabilties

Linux capabilities are a fine-grained security mechanism that splits the traditional superuser (root) privileges into smaller, more manageable pieces. Instead of giving a process full root access, Linux capabilities allow for the assignment of specific privileges to processes, such as the ability to bind to a network port or change the system time, without granting full administrative control. This is particularly useful for increasing system security, as it limits the potential damage caused by vulnerabilities in applications. For example, a web server might require the ability to bind to port 80 but should not need full root privileges. By assigning only the necessary capabilities to a process, the security surface area is reduced, making it harder for an attacker to exploit. This approach is commonly used in containerized environments, where processes are run with just the capabilities they need to function.

2.10 Seccomp (Secure Computing Mode)

Seccomp is a security feature in the Linux kernel that allows administrators to filter and restrict the system calls that processes can make. It provides a mechanism to limit the actions that a process can perform, which is particularly useful in reducing the attack surface of applications, especially in containerized environments. With seccomp, you can create a whitelist or blacklist of system calls, effectively controlling which parts of the operating system a process can access. For instance, you could block potentially dangerous system calls like execve, which is used to execute programs, or restrict the ability to open files by denying access to certain file descriptors. This reduces the risk of an attacker exploiting a process to gain control over the system. Seccomp is often used in combination with other security mechanisms like AppArmor and SELinux to provide an additional layer of protection.

2.11 App Armor (Application Armor)

AppArmor is a Linux security module that provides Mandatory Access Control (MAC) by enforcing security policies on individual programs. It restricts the actions that applications can perform by defining profiles that specify which files, directories, and capabilities the program can access. AppArmor operates by using a set of predefined or custom profiles, which are attached to programs, to limit their access to system resources. For example, an AppArmor profile for a web server might restrict the server's access to specific directories and prevent it from executing arbitrary binaries or making network connections to external IP addresses. AppArmor is easier to configure and manage than some other security modules, and it provides a level of protection by ensuring that even if an application is compromised, the attacker has limited access to the system.

2.12 selinux

SELinux is a security module that provides more granular control than traditional discretionary access control (DAC) by using mandatory access control (MAC). It enforces policies that define which processes or users can access specific files or resources, based on security contexts. These contexts are applied through SELinux policies, which restrict access even if a process has standard permissions. SELinux is commonly used in high-security environments, such as government or financial systems, to prevent unauthorized access. While powerful, it requires careful configuration to avoid restricting legitimate processes and reduce the risk of privilege escalation.

2.13 YAMA

Yama is a Linux Security Module (LSM) that adds additional restrictions to the ptrace system call, which is commonly used for debugging and process tracing. While ptrace is a powerful tool for legitimate uses like debuggers and profilers, it can also be abused by malicious processes to interfere with or spy on other running processes. Yama was introduced to mitigate these risks by controlling and limiting which processes can use ptrace on others, providing a simple but effective hardening mechanism against certain types of attacks.

The core of Yama's functionality revolves around the /proc/sys/kernel/yama/ptrace_scope setting, which controls how restrictive the kernel is regarding ptrace access. The available levels range from 0 to 3, each enforcing progressively stricter controls. With ptrace_scope=0, the system behaves traditionally, allowing any process to ptrace another process owned by the same user if the target is marked as dumpable. At 1, only parent processes can trace their children, limiting the risk of arbitrary tracing. Level 2 requires a process to have CAP_SYS_PTRACE capabilities (effectively root-level permissions) to trace any process, and level 3 completely

disables ptrace system-wide, locking down the system from process tracing even by privileged users.

Yama is especially valuable in multi-user systems or environments where processes run under the same user account but should remain isolated from one another, such as in containerized or sandboxed environments. While it is a relatively lightweight security module compared to more comprehensive systems like SELinux or AppArmor, it adds a meaningful layer of defense against local privilege escalation attacks. However, it's important to balance its restrictions with the need for debugging and profiling tools that rely on ptrace. In production environments focused on security, higher ptrace_scope values (such as 2 or 3) may be appropriate, while development systems typically keep it at 0 or 1 to allow standard debugging workflows.

Yama does not enforce its 'ptrace_scope' restrictions when attempting to trace host processes from within a container because Yama's enforcement is limited to the current process visibility defined by PID namespaces. In most container environments, the containerized process runs in an isolated PID namespace and cannot see or interact with host processes unless explicitly configured using options like '–pid=host'. Even when a container is started with '–pid=host', allowing it to see host processes, Yama primarily governs interactions within the same namespace context and is not designed to enforce security policies across different namespaces.

Additionally, for Yama to restrict tracing of host processes, the containerized process would not only need visibility of host PIDs but also require sufficient privileges, such as 'CAP_SYS_PTRACE', in the host's user namespace. In practice, other security mechanisms like AppArmor, SELinux, and seccomp often prevent such privilege escalation before Yama's policies even apply. As a result, Yama's effectiveness is primarily confined to preventing ptrace interactions between processes that are already within the same visibility scope, rather than serving as a cross-namespace isolation mechanism. This makes Yama a useful tool for hardening processes within the same

environment but not a comprehensive defense against container-to-host process tracing attempts.

The behavior of Yama is controlled by the `/proc/sys/kernel/yama/ptrace_scope` setting, which determines the level of restriction imposed on `ptrace` operations. The possible values are summarized below:

Table 2.1: Yama `ptrace_scope` Settings

Value	**Behavior**
0	No additional restrictions. Processes can trace any other process they have permissions for.
1	Restricted to child processes only. This is the default setting in many distributions.
2	Restricted unless explicitly allowed via `/proc/[pid]/uid_map` or using `prctl()`.
3	No `ptrace` allowed at all, even for processes with `CAP_SYS_PTRACE`. This is the most restrictive setting.

When Yama's ptrace_scope is set to 2, processes cannot freely attach to other unrelated processes, even if they have administrative capabilities or are running as root. In this mode, the ability to trace or debug a process must be explicitly granted by the target process itself or configured through system-level controls.

This permission can be granted using the prctl() system call with the PR_SET_PTRACER option, allowing a specific process to act as a tracer. Alternatively, the system administrator can modify the global ptrace policy by lowering the ptrace_scope value in /proc/sys/kernel/yama/ptrace_scope. Without such explicit authorization, ptrace attempts will consistently fail, regardless of user privileges or capabilities. This design enforces stronger security boundaries by preventing unauthorized inspection or manipulation of processes, even by privileged users, unless the target process explicitly consents to being traced.

Yama is a Linux Security Module (LSM) that enforces additional restrictions on the `ptrace` system call and process debugging mechanisms. Its primary purpose is to reduce the attack surface for exploits that rely on process introspection and

manipulation. By limiting the ability of processes to trace or interact with others, Yama enhances system security and makes privilege escalation attacks more difficult. This is particularly valuable in containerized environments and multi-user systems where process isolation is critical.

2.14 Kernel Implementation

Yama is implemented in the Linux kernel source file `security/yama/yama_lsm.c`. It integrates with the kernel's LSM framework by registering specific hooks that intercept and evaluate `ptrace` operations. During initialization, Yama registers its hooks using the following function:

```
void __init yama_add_hooks(void)
{
    pr_info("Yama: becoming mindful.\n");
    security_add_hooks(yama_hooks, ARRAY_SIZE(yama_hooks), "yama");
    yama_init_sysctl();
}
```

Here, `security_add_hooks` registers an array of hook functions defined in `yama_hooks`. These hooks include:

- `ptrace_access_check`
- `ptrace_traceme`
- `task_prctl`
- `task_free`

Each hook is associated with a specific kernel operation, allowing Yama to enforce its security policies at critical points.

2.15 Enforcement of `ptrace_scope`

The core of Yama's enforcement logic is implemented in the `yama_ptrace_access_check` function:

```
static int yama_ptrace_access_check(struct task_struct *child,
         unsigned int mode){
}
```

This function determines whether a `ptrace` operation should be permitted based on the current `ptrace_scope` value and the relationship between the tracer and tracee processes.

2.16 `ptrace_scope` Settings

Yama provides four levels of `ptrace` restrictions, configurable via `/proc/sys/kernel/yama/ptrace_`
These settings are summarized in Table 2.1.

2.17 Related Work

At the time of this writing, to our knowledge, Houdini is the first comprehensive approach for evaluating and comparing *container confinement mechanisms*. Much of the container security literature to date has focused on either vulnerability scanning of container images, building offensive/attack tools for container escapes, or proposing best practices for securing container runtimes. While these papers, tools, and documents do not directly share our research objectives, they still broadly fit into the container security landscape. Additionally, research on VM escapes and sandbox escapes has provided valuable insights into isolation vulnerabilities, many of which relate to container security. Techniques used to break out of hypervisors and sandboxed environments often share similarities with container escape methods, which

the importance of strong confinement mechanisms. The remainder of this section does not aim to exhaustively enumerate all container security tools and systems, but rather to highlight the various salient methods, including prior research on times the confinement of VMs and sandboxes have been escaped as it is related to work.

Regression Testing. One common practice in continuous integration/continuous deployment (CI/CD) pipelines is the use of automated vulnerability scanning tools to detect known vulnerabilities in container images. Tools such as Trivy [**?**] and Clair [**?**] are often integrated into CI pipelines to perform vulnerability assessments during the image build process. These tools automatically scan for security flaws in the container image, ensuring that only secure images are deployed to production. For example, GitLab's CI/CD pipeline includes built-in features for Container Scanning with Trivy, automatically running vulnerability scans whenever new Docker images are built or updated.

In addition to security scanning and policy enforcement, CI/CD pipelines also use container hardening tools like Docker Bench for Security [**?**] and Kube-bench [**?**] to ensure containers are securely configured. These tools can be integrated into the pipeline to automatically check container configurations against security best practices, such as ensuring that containers run as non-root users or that unnecessary ports are closed. CircleCI [**?**], for example, integrates Docker Bench to verify that new container builds follow best practices before they are pushed to production.

Vulnerability Scanners. Many free, open, and closed-source tools exist for identifying the presence of known vulnerabilities in container images. Shu et al. [**?**] developed a vulnerability scanning tool to scan Docker Hub container images at scale. They found that on average, official and community images have concerning amounts of vulnerabilities (180+), and that these vulnerabilities remain unpatched for hundreds

of days. Their study utilized the Docker Image Vulnerability Analysis (DIVA) framework, which automates the discovery, download, and analysis of over 300,000 Docker images. The findings revealed that more than 80% of both official and community images contained at least one high-severity vulnerability, and many images were not updated for extended periods. Moreover, vulnerabilities were often inherited from parent images to child images, further increasing security risks.

Best Practices. Users in search of (often high-level) advice on how to improve the security of their container deployments can consult one of many best practices guides available online. One such guide is the Center for Internet Security (CIS) Docker Benchmark [**?**]. In our experience, guides tend to offer generic advice (e.g., "*Only allow read access to the root filesystem*"). This is problematic because it oversimplifies container security. Generic advice may seem straightforward, but it doesn't account for the specific context or unique configuration of an individual deployment. Such one-size-fits-all recommendations can lead users to believe that simply following these broad guidelines is enough to secure their systems, potentially leaving critical vulnerabilities unaddressed. In reality, effective container security requires a nuanced, tailored approach that considers the particular needs and threat models of each environment. Houdini fills the gap between generic advice and practical security by enabling one to objectively evaluate the effectiveness of your container confinement measures.

Offensive Tools. The offensive security community has developed a range of specialized tools aimed at identifying and exploiting vulnerabilities within containerized environments. Containers, while providing strong isolation, are not immune to attacks, and these tools focus on testing the robustness of container defenses. CDK[1]

[1] `https://github.com/cdk-team/CDK`

and DEEPCE[2] are two widely used, open-source container penetration testing toolkits that leverage a collection of known exploits to gain persistence, escape the container, and gather sensitive information about the container environment. These tools aim to bypass isolation mechanisms like namespaces, cgroups, and SELinux, often using a variety of aggressive techniques to identify weaknesses in container security. CDK specifically uses any available method to circumvent security measures, providing a broad toolkit to test the limits of a container's defense. Meanwhile, DEEPCE, developed by stealthcopter, focuses on privilege escalation and container enumeration, attempting to exploit specific weaknesses to achieve container escapes, thereby compromising the host system. In contrast, Houdini takes a more systematic and structured approach to container security testing. Rather than focusing solely on exploiting vulnerabilities, Houdini allows for the direct comparison of defensive techniques by clearly defining the testing environment, outlining the exploit steps, and documenting the expected outputs.

Virtual Machine Escapes. VM escapes occur when an attacker successfully breaks out of a virtualized confined environment, gaining unauthorized access to the host system. Given that hypervisors manage multiple VMs on shared hardware, such escapes can lead to host compromise, cross-VM attacks, and lateral movement within cloud environments. One of the most notable VM escape vulnerabilities is VENOM (CVE-2015-3456) [**?**], which exploited a flaw in the virtual floppy disk controller (FDC) implementation in QEMU, affecting platforms like Xen, KVM, and VirtualBox. Other significant VM escape vulnerabilities include Cloudburst (2009), which exploited VMware's virtual graphics driver; CVE-2018-6981 and CVE-2018-6982, which targeted VMware ESXi, Workstation, and Fusion [**?**]; and multiple VirtualBox vulnerabilities (2017) enabling guest-to-host code execution [**?**]. Additionally,

[2] `https://github.com/stealthcopter/deepce`

side-channel attacks like MDS/ZombieLoad (2019) demonstrated how speculative execution flaws could be leveraged across VMs [**?**]. More recent research has uncovered weaknesses in encrypted virtualization, such as the SEVered attack (2018), which extracted plaintext memory from AMD SEV-protected VMs [**?**], and the Heckler attack (2024), which manipulated non-timer interrupts to compromise confidential VMs in AMD SEV-SNP and Intel TDX [**?**].

Sandbox Escapes. Sandboxing is a critical security mechanism for restricting the execution of untrusted code, commonly used in browsers, mobile apps, and security tools. A sandbox escape occurs when an attacker bypasses these restrictions, executing arbitrary code with higher privileges and potentially leading to full system compromise. Modern web browsers heavily rely on sandboxing to isolate JavaScript execution from the underlying OS. However, several high-profile vulnerabilities have enabled sandbox escapes. For example, CVE-2019-5786 [**?**], a use-after-free vulnerability in the FileReader API of Google Chrome, allowed attackers to escape Chrome's sandbox and execute code on the host. Other notable sandbox escape vulnerabilities include CVE-2022-26706, which exploited a flaw in macOS's App Sandbox to execute code outside of restricted permissions [**?**]; CVE-2023-36719, a stack corruption vulnerability in Windows OS that enabled escape from the Chromium sandbox [**?**]; and CVE-2024-4040, which was actively exploited to break out of CrushFTP's Virtual File System (VFS) [**?**]. Additionally, vulnerabilities in Judge0, an online code execution system, exposed systems to full takeover by allowing attackers to escape its sandbox [**?**]. Research has also uncovered critical flaws in vm2, a JavaScript sandbox library, which could be exploited to bypass its security restrictions and gain remote code execution [**?**].

2.17.1 Apparmor

AppArmor is a Linux security framework based on mandatory access control, similar to SELinux, but it focuses on restricting individual programs [9]. It works by applying predefined security profiles that limit what a program can do [9]. In Docker, AppArmor can be used to load these profiles into containers to restrict their behavior. If no profile is specified, Docker automatically applies a default one [9].

Mattetti et al. [65] developed LiCShield as a framework that uses AppArmor as its enforcement backend. It automatically generates custom AppArmor security profiles by analyzing container behavior in a pre-production environment. These profiles are then used to tightly confine both the container's runtime behavior and its interactions with the host system, ensuring that only authorized actions are permitted during execution.

Sun et al [13] extend AppArmor to work independently inside containers by introducing a new concept called a security namespace. Normally, AppArmor is global, it enforces one set of security rules for the whole system, which means containers cannot define or apply their own AppArmor policies. The authors solve this problem by modifying the Linux kernel so that each container can have its own separate AppArmor namespace. This allows a container to load and enforce its own AppArmor profiles for its applications, independent of the host system's policies.

With this change, both the host and the container can apply AppArmor protections at the same time. For example, while the host might enforce a general container confinement policy, the container itself can load a specific profile to restrict what Firefox or MySQL can do [13].

Zhu et al [9] create Lic-Sec, which is an enhanced AppArmor profile generator that improves Docker container security by combining the strengths of Docker-sec and LiCShield. The reason for creating a similar tool to LiCShield is that LiCShield is incompatible with recent versions of Docker. Lic-Sec automatically creates fine-

grained profiles based on container behavior, restricting capabilities, network access, file operations, mounts, and shell execution. By using both Auditd and SystemTap for tracing, Lic-Sec generates stricter, more comprehensive profiles that block privilege escalation and container escape attacks. It adapts to container use cases and enforces these profiles at runtime for stronger and automated protection.

Houdini could have influenced Zhu et al [9] work by providing an automated framework to evaluate Lic-Sec's effectiveness across diverse environments. For example, instead of manually testing exploits against a fixed kernel version, docker version and set of container images, the authors could have used Houdini to run the same exploit across multiple kernel versions, Docker versions, configurations, and AppArmor profiles to identify gaps in enforcement. Additionally, Houdini's ability to replay tricks across time and machines would have allowed for more reliable validation of Lic-Sec's defenses, such as verifying whether blocking /bin/bash consistently prevents privilege escalation across real-world images.

2.17.2 Control Groups

cgroups (short for control groups) are a Linux kernel feature used to manage and limit system resources like CPU, memory, and I/O for groups of processes [72]. They allow administrators or container runtimes like Docker to control how much of a resource each container can use. For example, Docker can apply cgroup settings to restrict a container's CPU time or memory usage. This helps prevent a single container from consuming too many resources and affecting others on the system [72]. While cgroups are not designed to stop one container from accessing or interfering with another container's data or processes [72], they are essential for defending against certain types of Denial-of-Service (DoS) attacks by limiting the damage a container can cause through resource exhaustion [72].

Chelladhurai et al. [72] identifies DoS attacks as a significant threat in container-

ized environments like Docker. The authors emphasize that while containers offer process isolation, they lack strong guarantees against resource abuse, which can allow a malicious container to exhaust host resources and impact other containers.

To address this, they propose several practical mitigation strategies using Linux cgroups [72]. They argue that cgroups are essential for defending against DoS attacks by limiting a container's access to CPU, memory, and I/O resources. However, they also note that cgroups alone do not provide full isolation between containers. Their proposed DoS mitigation technique includes three test cases: setting memory limits directly in the docker run command using -m=512mb, applying default memory limits in the Docker configuration file (/etc/default/docker), and enforcing a system-wide memory limit for containers that do not specify their own [72]. For evaluation, the paper includes sample attack code to simulate DoS scenarios. One example allocates memory indefinitely using malloc, while another consumes CPU resources by endlessly calculating prime numbers. These tests demonstrate how resource limits can effectively prevent containers from overwhelming the host.

The original paper takes a manual approach to testing DoS attacks by running memory and CPU-intensive scripts inside containers with various resource limits. While this method demonstrates the effectiveness of using cgroups to mitigate DoS risks, it is limited in scale and reproducibility. Houdini can automate and greatly expand this type of testing. Instead of running scripts manually, Houdini can encode these attacks as repeatable tricks, such as infinite memory allocation or continuous CPU-bound computations. These tricks can then be executed systematically across a wide range of Docker versions, kernel versions, and container configurations (e.g., with or without memory limits like the -m flag). This is especially important because the implementation and enforcement behavior of cgroups can differ between kernel versions, particularly between cgroups v1 and v2, or in how specific resource limits are handled. By capturing and comparing results, Houdini enables a scalable and

repeatable evaluation framework. It can identify which combinations of settings are vulnerable to DoS and which configurations effectively block the attacks, providing a more comprehensive assessment than manual testing.

2.17.3 Extended Berkeley Packet Filter

eBPF (extended Berkeley Packet Filter) is a Linux kernel technology that allows small, sandboxed programs to run safely inside the kernel without modifying kernel source code or loading kernel modules. Originally designed for network packet filtering, eBPF has evolved into a general-purpose tool for tracing, monitoring, and security enforcement. In essence, eBPF lets developers attach custom logic to various kernel events, like system calls, network activity, or file operations, and collect data or enforce rules with minimal performance overhead. Because eBPF programs run in a restricted environment and are verified for safety before execution, they offer a powerful but secure way to extend kernel behavior dynamically.

Lopes et al. [55] use eBPF as the core mechanism for tracing system calls made by containerized applications. The goal of the system is to automatically generate custom Seccomp profiles that strictly limit the syscalls a container can perform, thereby reducing the attack surface. To achieve this, the authors use eBPF's powerful kernel-level tracing capabilities to hook into every syscall. Specifically, they implement a BCC (BPF Compiler Collection) program that attaches eBPF probes to all system calls and logs detailed information such as the syscall name, timestamp, process ID, and the namespace from which the syscall originated.

For Lopes et al. [55], the goal is to automatically generate Seccomp profiles by tracing the actual system calls used by a containerized application. While their approach uses eBPF for runtime tracing, Houdini could have provided complementary value by automating and scaling this profiling process across many container configurations, Docker versions, and kernel versions. Houdini's test matrix model allows

systematic exploration of diverse runtime flags (e.g., –privileged, –cap-add, AppArmor profiles) to evaluate whether the generated Seccomp profile remains effective across different settings. Additionally, Houdini could validate whether the reduced syscall profile indeed blocks known exploits, as done in Lopes' Apache test case, but at scale.

This namespace information is crucial because it allows the system to filter out irrelevant syscalls made by the host or other processes, ensuring that only syscalls from the target container are recorded. These logs are then processed to generate a Seccomp profile that whitelists only the syscalls actually used by the application during testing. One of the key advantages of using eBPF is its runtime-agnostic nature. It works not only with Docker but also with Podman or any other container technology, since the tracing happens at the kernel level. Additionally, eBPF's integration into the CI/CD pipeline enables automatic profile generation whenever code changes are committed, making it easy to keep Seccomp profiles up to date with minimal manual effort. As a proof of concept, the authors applied their system to a vulnerable Apache container and demonstrated that, after tracing and generating a Seccomp profile using eBPF, the container was no longer susceptible to a known RCE vulnerability (CVE-2018-11776).

He et al. [59] found a new type of attack where eBPF can be misused to break container isolation in shared-kernel environments like the cloud. eBPF was designed to safely let programs interact with the Linux kernel for things like monitoring and debugging. But the paper shows that attackers inside a container can use eBPF helper functions, such as ones that read or write other processes' memory to spy on or interfere with programs running outside the container, including on the host. These attacks can steal sensitive data, crash important services like systemd, or even escape the container by taking over a root-level process on the host.

One key problem is that eBPF tracing programs are not limited by namespaces,

so they can see and interact with processes system-wide. The authors tested this in real-world services like Jupyter notebooks and Kubernetes clusters and successfully escaped containers or spread attacks across nodes. Even trusted tools like Cilium and Datadog that use eBPF for monitoring could be at risk if used in containers with the wrong settings.

To fix this, the paper proposes a new system called CapBits. It gives each process fine-grained control over which eBPF features it can use or be targeted by. CapBits is faster and more secure than previous methods and can stop attacks where eBPF is used to hijack sensitive processes. Overall, the paper warns that while eBPF is a powerful tool, its risks in container environments are not well understood and need better protection.

Jiao et al. [56] introduce Seccomp-eBPF, a new way to improve system call filtering on Linux using the extended Berkeley Packet Filter (eBPF). System call filtering is a key technique used to protect the operating system from untrusted applications, such as containers, by controlling which system calls are allowed. The current standard, Seccomp with classic BPF (cBPF), is widely used but has serious limitations. It only allows basic, static checks and cannot store information or respond to complex patterns of behavior, making it hard to implement advanced security policies.

Seccomp-eBPF solves this by allowing developers to write more powerful and flexible filters using eBPF, a newer language already used in many parts of the Linux kernel. With Seccomp-eBPF, filters can track state (such as counting how many times a system call has been used), detect specific sequences of system calls, prevent race-condition attacks by forcing certain system calls to run one at a time, and safely examine data passed from user programs. These features make it possible to block sophisticated attacks that cBPF cannot handle.

The authors implemented this system in the Linux kernel and added support for it in the crun container runtime. They showed that it works well in practice. For

example, it reduced the number of allowed system calls during the startup of common server applications by up to 55%, significantly shrinking the attack surface. Despite its added flexibility, Seccomp-eBPF maintained performance similar to optimized cBPF filters and was much faster than user-space alternatives like Seccomp Notifier. Overall, Seccomp-eBPF offers a safer and more powerful way to secure Linux systems, especially in environments like containers that rely heavily on system call filtering.

Findlay et al. [1] use eBPF to build a new security system called BPFCONTAIN that improves container security on Linux. Containers often rely on tools like seccomp, AppArmor, and SELinux for isolation, but these tools can be complicated, inconsistent, and not designed specifically for containers. To solve this problem, BPFCONTAIN uses eBPF to monitor and control what containers are allowed to do directly inside the Linux kernel. It checks actions like file access, network use, and system calls, and blocks anything not allowed by the container's policy. These rules are written in a simple YAML file, making it easy to manage. BPFCONTAIN loads these rules and programs at runtime, without changing the Linux kernel or container tools like Docker. It also adds extra protections to block known container attacks, such as privilege escalation.

2.17.4 System Call Filtering

System call filtering is a security technique used to control which system calls a program is allowed to make to the operating system. Since system calls are the primary way that applications interact with the kernel, restricting them helps reduce the attack surface and prevent malicious behavior. On Linux, this is commonly done using Seccomp, which lets developers create filters to allow or block specific syscalls. Traditional filtering with classic BPF (cBPF) is simple and efficient but limited, it only supports static rules and cannot track complex behavior or context. More advanced techniques, such as those using eBPF, improve flexibility by allowing filters

to maintain state, inspect arguments, and respond dynamically to syscall patterns.

This paper used system call filtering as a key part of its security monitoring approach. Gantikow et al. [63] used two tools named Sysdig and Falco that rely on a kernel module to capture and analyze system calls made by containers. By watching these low-level operations, the system can spot unusual or potentially harmful behavior. Sysdig collects all system calls and lets users filter them by container, process, or syscall type to reduce noise. Falco adds a layer of security by letting users define rules that flag suspicious behavior based on these system calls.

For example, the authors created rules to detect when a container tries to access files outside its expected directories using the open() syscall. They also detected unauthorized programs being run inside containers by monitoring execve calls. Attempts to break out of containers were flagged by watching for the setns syscall, and unexpected network connections were detected by tracking relevant network-related system calls.

While Falco can only detect individual suspicious events, not patterns or sequences of events like those involved in buffer overflows or denial-of-service attacks, the use of system call filtering still proved effective in catching many types of threats with low performance impact.

Ghavamnia et al. [27] present Confine, a system that automatically generates restrictive system call policies for Docker containers. Instead of relying solely on dynamic analysis (which often misses rare or conditional execution paths), Confine uses a hybrid approach: it performs a short dynamic analysis to discover all executables launched during container startup and then uses static analysis on those binaries to extract the complete set of system calls they may invoke. This includes both libc-invoked and directly invoked syscalls.

The goal of this syscall filtering is to allow containers access only to the syscalls they truly need, thereby blocking access to the rest, many of which may contain

vulnerabilities or provide attack vectors for privilege escalation or container escape. By creating custom Seccomp BPF profiles that deny all unnecessary syscalls, Confine ensures that even if an attacker gains access to a container, they cannot exploit most of the kernel interface.

The evaluation showed that Confine could disable 145 or more system calls (out of 326) for over half of the tested containers, neutralizing 51 known kernel CVEs. This significantly exceeds Docker's default Seccomp filter, which only disables 49 syscalls.

Houdini could be used to dynamically test whether Confine's generated Seccomp profiles are too restrictive or too permissive. For instance, if Houdini can successfully run an attack trick under a Confine policy, it might indicate an over-permissive profile. Conversely, if legitimate workloads fail under Confine's policy in Houdini's environment, this suggests over-restriction.

Confine assumes that policies generated statically will remain correct across systems. But Linux syscall implementations and Docker behavior change over time. Houdini could systematically evaluate if the same Confine-generated profile works consistently across kernel versions, and flag new vulnerabilities.

Jiao et al. [56] used system call filtering to improve the security of Linux systems by controlling which system calls a program is allowed to make. Normally, this is done using a tool called Seccomp, which filters system calls based on simple rules written in a limited language called cBPF. However, cBPF is very basic, as it can only check the syscall number and some simple arguments. It cannot remember past behavior, check syscall sequences, or read memory safely. This makes it hard to write strong security rules.

Jiao et al. [56] solve this by creating a new system called Seccomp-eBPF, which uses the more powerful eBPF language instead of cBPF. eBPF allows writing filters that can remember past system calls, count how many times a system call was used, block system calls if they're too frequent, or only allow them in certain program

phases. It can also safely inspect data passed by pointer (like file paths), and prevent known bugs caused by racing system calls.

With these improvements, Seccomp-eBPF can block real-world attacks that cBPF cannot. It also helps reduce the number of system calls allowed in early stages of a program (reducing attack surface) by over 50% in some cases. The paper shows that this system is both safer and faster than older methods that rely on user-space monitoring, and it works well in environments like containers.

2.17.5 Rule-based Systems

A rule-based system is one that makes decisions or detects events based on a predefined set of rules. These rules are usually written by security experts or administrators and follow simple if-then logic. For example, a rule might state that if a certain system call is made by an unexpected process, the system should flag it as suspicious or block it. This approach allows for clear, predictable behavior because every decision is based on explicitly defined conditions.

In the context of container or system security, rule-based monitoring is often used to detect abnormal behavior. Tools like Falco apply rules to system call traces to identify activities that deviate from what is considered normal or safe. For example, rules can be written to detect unauthorized file access, unusual network activity, or attempts to escalate privileges. These systems are easy to understand and customize, making them widely used in production environments.

However, rule-based systems also have limitations. They can only detect behaviors that have been anticipated and written into the rules. This makes them less effective against new or complex attack patterns that do not match existing rules.

A proposed method by Jian et al. [6] implemented as a shell script that runs on the host and follows a sequence of fixed rules to identify suspicious activity. It first collects metadata about all running containers and their processes, including each

process's PID, name, and namespace tag (Nsid). Then, it compares the namespace tag of each container process against the namespace tag of that container's init process. According to the authors, if a process inside a container has a different namespace tag than its init process, it likely indicates that the process has escaped its namespace—an early sign of a container escape attempt.

This comparison logic is rule-based because it applies a deterministic rule: if Nsid(process) does not equal Nsid(init), then mark the process as suspicious. Further, the script checks whether the suspicious process resembles a shell (based on the process name). If both conditions are met, the system takes action: it kills the process and tries to trace the attacker (e.g., by retrieving the user ID or IP address).

Houdini could have helped improve the work by Jian et al. [6] by making the testing more thorough, automated, and easier to repeat. In the paper, the authors propose a way to detect Docker escape attacks by checking if a process inside the container has a different namespace tag than the container's main process. They test this using only two known attacks: one involving the recvmsg system call and another using the Dirty COW vulnerability.

With Houdini, the authors could have gone much further. Houdini can automatically run many different escape tricks, both known and custom-made across a wide range of Docker versions, Linux kernel versions, and settings like user privileges, Seccomp filters, and namespace configurations. This would help find out whether the detection method works well in more situations or if it fails in certain cases.

Gantikow et al. [63] propose a rule-based monitoring system that detects security violations in containerized workloads by analyzing system call activity. This is accomplished using two open-source tools: Sysdig and Falco. Sysdig provides a kernel module that captures all system calls made by containers, while Falco builds upon this by applying predefined rules to flag suspicious behavior. These rules are written in a declarative format and can describe conditions such as unauthorized file access,

unexpected application launches, or privilege escalation attempts.

In this rule-based model, each rule defines a pattern of behavior that is considered abnormal or malicious. For example, a rule might detect a container process trying to access files outside its designated working directory (indicating a possible information leakage or path traversal attack), or flag execution of unknown binaries inside a container (which might imply a crypto-miner or reverse shell). Rules can be whitelist-based (defining what is allowed) or blacklist-based (defining what is forbidden), and are matched against real-time system call events.

The paper presents various attack scenarios like container breakout, unauthorized networking, or loading kernel modules, and shows how each can be detected with carefully written Falco rules. While the rule-based approach is effective for many types of misuse, the authors acknowledge limitations. Notably, Falco cannot detect frequency-based or multi-event anomalies (e.g., Denial of Service or buffer overflows), as it only supports detection of single-event anomalies.

the paper by Brady et al. [51] uses a rule-based system as part of its approach to securing Docker containers. The authors incorporate Anchore Engine into their CI/CD pipeline, which applies a set of predefined security rules, also known as policies, to each Docker image. These rules include checks for known security vulnerabilities and for unsafe Dockerfile configurations, such as whether the image exposes port 22. In their implementation, the authors use Anchore's default rules, but they note that developers can create and apply their own custom rules based on specific security needs. This rule-based system allows the pipeline to automatically detect and block images that do not meet the defined security standards. It helps developers identify and fix issues early in the development process, improving the overall security of containerized applications.

The paper by Zhu et al. [9] describes three tools Docker-sec, LiCShield, and the proposed Lic-Sec all of which rely on generating and enforcing rule-based security

profiles using AppArmor. These profiles consist of explicit allow/deny rules that define what operations a containerized application can perform, such as file accesses, network operations, and use of Linux capabilities.

Docker-sec uses dynamic monitoring to learn what capabilities and network accesses a container actually uses, then generates an AppArmor profile with rules that only permit those actions. LiCShield, on the other hand, uses SystemTap to trace kernel-level operations and generates rules for more types of actions, such as file mounts and execution paths. Lic-Sec combines both tools to generate a more comprehensive rule set that includes deny rules for unnecessary shell commands and tighter restrictions on mounts and execution behavior.

In all cases, the tools use observed behavior during a training phase to produce explicit, rule-based security profiles that constrain container behavior at runtime. These rules are enforced by the AppArmor Linux Security Module, making this a clear example of a rule-based system in container security.

The paper by Chen et al. [68] uses a rule-based system as part of its SEAF framework to detect security flaws in Docker container images. SEAF defines a set of detection primitives that act as basic operations for writing flexible detection rules. These primitives include content matching, feature matching, data recording, usage frequency analysis, and file extraction. Each type of vulnerability, such as misconfiguration, unauthorized access, sensitive information leakage, malware, and software vulnerabilities, is detected by combining specific primitives into rules. For example, to detect unauthorized access in Redis containers, SEAF extracts configuration files and uses regular expressions to search for insecure settings like missing passwords. These rule combinations allow SEAF to systematically inspect container layers and identify flaws by applying consistent logic across images. This rule-based approach is modular, meaning new rules can be added easily to address emerging threats without changing the underlying system. As a result, SEAF can adapt to various application

scenarios and detect a wide range of security issues using well-defined and reusable rule patterns.

2.17.6 OS Changes

Luo Yang et al. [26] investigates how containers can still leak information through covert channels, even when isolation mechanisms are in place. It focuses on Docker and shows that the default security features such as namespaces and cgroups are not enough to prevent secret communication between containers. These covert channels take advantage of shared physical resources, allowing containers to bypass the expected security boundaries.

The authors present five high-capacity covert channel techniques. These include CPU load modulation, cache-based covert channels, disk I/O delay signaling, memory bus contention, and file lock contention. Each method allows a sender container to encode data into patterns of resource usage. The receiver container then decodes this data by observing how these patterns affect performance. For example, a sender might create bursts of CPU activity to represent binary data, while the receiver measures CPU latency to interpret the signal. These attacks are a form of side-channel communication, where indirect changes in system behavior are used to leak information.

The paper describes scenarios where a sender container transfers hidden information to a receiver container, despite there being no authorized communication path between them. This demonstrates that containers can communicate by exploiting shared hardware such as CPU caches or memory bandwidth. Even without network access or shared filesystems, the sender can affect system behavior in ways that the receiver can detect and interpret.

In this paper, Sun et al. [13] introduce the idea of security namespaces, a new feature in the Linux kernel that lets each container have its own security settings

using tools like AppArmor and IMA. Normally, these tools are set by the host and apply to all containers. This means containers can't create or manage their own security rules, which is a problem when you want containers to be more isolated or run different policies. The paper's solution is to separate these security settings so that each container can have its own, without affecting the host or other containers.

To make this work, the authors made small changes to the Linux kernel, only about 1,000 to 1,500 lines of code. They tested their changes in two ways. First, with IMA, containers were able to set their own policies, like only allowing signed programs to run. The containers correctly blocked unsigned or changed programs, and they couldn't get around the host's security rules, which stayed in place. Second, with AppArmor, they tested 20 common programs that already had security profiles. Sixteen of them ran correctly inside containers using both the container's and the host's security rules. For the four that didn't, the system gave warnings ahead of time to avoid crashes.

Even though this new design is useful, it doesn't work with systems like Houdini. Houdini depends on the way Linux currently handles security, where rules like AppArmor and IMA are shared across the whole system. If Houdini had to work with these new security namespaces, it would need to be redesigned from the ground up. Because of that, this paper's approach, while creative and effective, isn't compatible with how Houdini is built today.

The paper by Sun et al. [13] relate to OS changes because it shows that even though microVMs are meant to be isolated, some of their operations are still handled by the host operating system, creating security risks. The paper finds that parts of the system such as virtiofsd and kernel threads like kvm pit and vhost are not fully isolated. They run with too much access or are not limited by cgroups. To fix this, the authors suggest changing how the OS manages these components by placing kernel threads into the correct cgroups and reducing unnecessary permissions.

2.17.7 Secure Computing

Lopes et al. [55] present a method to improve container security by automating the creation and maintenance of Seccomp profiles. Seccomp is a Linux kernel feature that limits the system calls a containerized application can make, significantly reducing the potential attack surface. However, maintaining these profiles is difficult because it requires a detailed understanding of which syscalls are needed and must be updated whenever the application changes. To address this, the authors develop a system that integrates with CI/CD pipelines to automatically trace the syscalls made during unit and integration testing using eBPF. These syscalls are collected, and a custom Seccomp profile is generated that whitelists only those calls, blocking everything else by default. For applications with incomplete test coverage, the system can use fuzzing to stimulate additional syscall paths. The approach ensures that each container has a tailored Seccomp profile, improving security without manual configuration. The authors show that this method can mitigate real-world vulnerabilities, such as CVE-2018-11776, by blocking the syscalls required for exploitation.

Ghavamnia et al. present a static analysis-based system named Confine that generates restrictive system call policies for Docker containers. Unlike previous solutions that rely on dynamic training with realistic workloads, which can miss rarely executed paths, Confine uses static analysis of container binaries and their dependencies to identify the complete superset of system calls required for correct operation. This allows it to generate Seccomp (Secure Computing Mode) profiles that deny all other system calls, significantly reducing the container's kernel attack surface.

The authors evaluated Confine on 150 Docker images and found that it could disable over 145 system calls out of 326 for more than half the containers, far more than Docker's default Seccomp policy. This reduction effectively neutralized 51 known kernel CVEs, demonstrating strong security gains without breaking application functionality. The approach also handles direct syscall invocations, language-specific system

call mechanisms such as Go and Java, and dynamic library loading.

Confine directly builds on Seccomp as its enforcement mechanism. It generates custom Seccomp BPF profiles that define allow and deny lists for system calls per container. By statically analyzing what syscalls are truly necessary, Confine tailors these Seccomp profiles to the exact needs of each container, minimizing the set of syscalls exposed to potentially malicious applications and enhancing the host's protection. This use of Seccomp is central to the paper's method for reducing kernel attack surfaces and mitigating container escape vulnerabilities.

Chapter 3

Design

The design of Houdini is guided by a set of core principles aimed at enabling reliable, large-scale testing of container security confinement. At its core, Houdini seeks to ensure **reproducibility**, so that each test yields the same result when run under identical conditions, and behaves consistently across time and environments. This is achieved by pinning exact versions of all key components—such as the kernel, Docker, and runc—and executing each test inside a clean, stateless VM.

Alongside reproducibility, Houdini is also built for **scalability**. Despite running one test at a time, the framework can execute large test matrices automatically, without requiring users to script each case manually. Instead, users define test configurations declaratively, and Houdini expands them into all valid combinations and runs each as an isolated test case. This makes it easy to scale experiments across hundreds or thousands of container configurations with minimal effort.

It also enforces strong **host isolation** by executing potentially dangerous container workloads inside tightly controlled VMs, preventing any impact on the host system. The framework supports **flexibility across configurations**, allowing users to test a wide range of Docker flags, Linux capabilities, AppArmor profiles, and runtime versions. It emphasizes **performance** by reducing overhead through lightweight

VMs and precompiled binaries. Finally, Houdini prioritizes **transparency and debuggability** by logging all configuration inputs and test results in a structured format, enabling users to trace and interpret outcomes. The rest of this chapter describes how each of these design goals are achieved in Houdini.

3.1 System Architecture

Houdini is a modular framework designed to enable secure and reproducible testing of container confinement across diverse configurations. Its architecture is divided into host-side and guest-side components, each with a specific and isolated role. This separation of responsibilities ensures strong test isolation, determinism, and ease of scaling to large test matrices.

On the host side, users interact with Houdini through a lightweight Python-based command-line interface (CLI). The CLI is responsible solely for parsing user input, such as selecting a trick, running a VM, or building the Buildroot-based filesystem. It does not manage execution logic itself. Once the user provides input to run a trick, the CLI delegates control to a programmatic API, which is responsible for managing the execution flow.

In Houdini, a single test case is referred to as a *trick*, which encapsulates a specific container security scenario. Each trick consists of three parts: a configuration file, a Dockerfile, and an executable script. The configuration file defines how the container should be launched, including its security parameters and references to the other two components. The Dockerfile specifies how to build the container image, installing dependencies and copying the script. The trick script (which can be written in any buildroot-supported language) contains the logic for performing the security test, such as attempting to read a protected file or escalate privileges. It is expected to emit a clear "pass" or "fail" message based on whether the action succeeds.

The API communicates with a Flask-based server that manages the lifecycle of a QEMU-based VM. Each trick is executed in a newly booted VM, built from a minimal, stateless image generated by Buildroot. This clean-slate approach ensures that every test runs in a predictable and reproducible environment, free from side effects caused by previous tests or host-level conditions. No persistent data is shared between executions, and the guest environment is intentionally stripped down. No background daemons, user services, or external dependencies exist inside the VM unless explicitly configured by the test.

The Flask server sends the test configuration and trick information into the VM via an HTTP request. Inside the guest system, a listener process continuously waits for such requests. When a test arrives, the listener prepares the environment by building a Docker image using the specified Dockerfile, and launches a container with the defined security parameters. These includes, but is not limited to Linux capabilities, namespace settings, seccomp profiles, AppArmor rules, volume mounts, and user privileges. The container is then launched with these constraints, ensuring that the environment inside aligns exactly with the configuration described by the user.

Once the container is running, the trick script is automatically executed. This script could perform a targeted security check, such as reading protected files, sending signals across namespaces, or attempting privilege escalation, and prints a clear "pass" or "fail" message to standard output. The listener captures the output, exit code, and any standard error, and sends this result data back to the host's Flask server. On the host, the result is logged in a `.log` file. This enables precise traceability, post-run analysis, and side-by-side comparison of multiple configurations.

Figure 3.1 illustrates the flow of information between Houdini's components, from user input on the CLI to trick execution inside the guest VM, and result collection on the host.

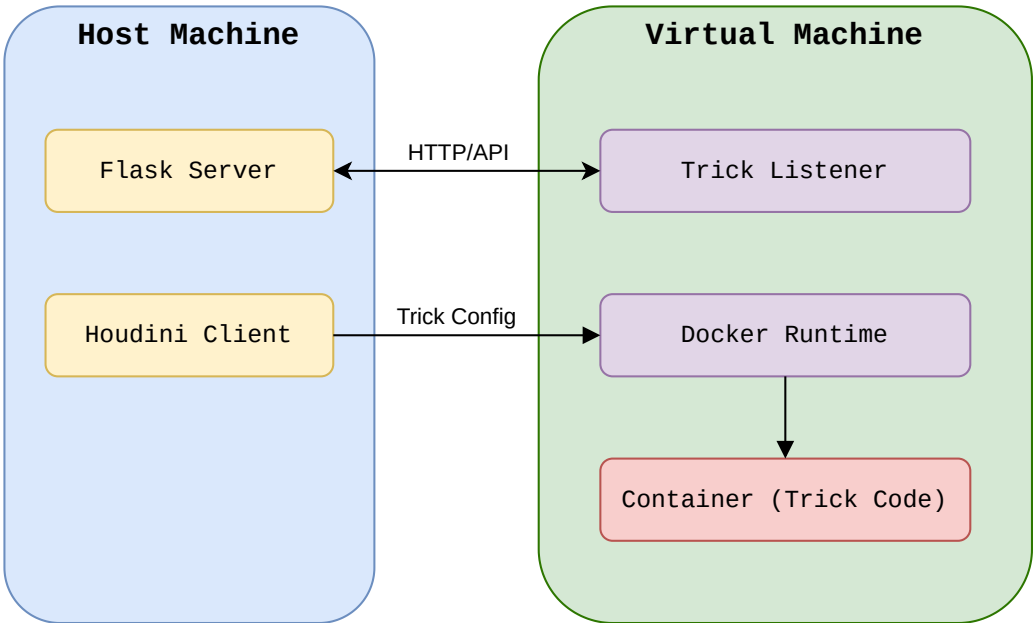


Figure 3.1: Architecture of Houdini

3.2 Reproducibility

A central design goal of Houdini is to ensure that container security tests are reproducible, meaning that the same test, when run under the same conditions, will produce identical results regardless of when or where it is executed. Houdini achieves reproducibility through a combination of strict version control, deterministic execution environments, and isolation from external influences.

Each test in Houdini runs inside a new VM that is freshly booted from a Buildroot-based image. This means that every test starts in a clean environment, with no leftover data or settings from previous runs. As soon as a test finishes, the VM is thrown away, so nothing carries over into the next test. This helps make sure that results are consistent and not affected by what happened before.

The Buildroot image is very minimal, it only includes what's absolutely needed to run the test. It doesn't have background programs like update services, SSH, or scheduled tasks, which could introduce randomness or unexpected behavior. By keep-

ing things simple and stripped down, Houdini ensures that the only things running in the VM are the ones the user specifically added for the test.

This clean and predictable setup makes it much easier to get the same results every time a test is run. Even if a trick is repeated many times or on different machines, the outcome stays consistent because the environment is always the same.

To avoid changes caused by updates or host system drift, Houdini uses a version pinning mechanism. Version pinning refers to the practice of explicitly specifying and using fixed versions of software components rather than relying on the latest or system-default versions. This ensures that tests always use the same binaries, regardless of when or where they are run, and prevents unexpected changes in behavior due to automatic updates or upstream changes. In the context of container security testing, version pinning is essential because even minor updates to Docker, runc, or the kernel can introduce subtle behavioral differences.

To ensure reproducibility, Houdini enforces strict control over the versions of all critical software components involved in test execution. Critical software components include the Linux kernel, Docker, and runc. Container behavior can change significantly between versions of these tools, even when the container configuration remains unchanged. To eliminate variability caused by software drift, Houdini adopts a comprehensive version pinning strategy.

Each test in Houdini explicitly declares the versions of Docker, runc, and the Linux kernel to be used. These components are downloaded from official release sources, precompiled, and stored in a structured local cache. This guarantees that the same binaries are reused across repeated runs. For example, if a trick is intended to evaluate a vulnerability in runc 1.0.0-rc2 but verify its resolution in 1.1.12, Houdini retrieves the exact binaries for each version and uses them directly, without relying on system package managers or external internet access. Similarly, multiple Docker versions such as 20.10.3, 23.0.1, or 27.0.3 can be tested against the same logic to study

behavioral drift.

Houdini uses an overlay filesystem to make testing both efficient and consistent. Instead of rebuilding the entire VM image from scratch for every test case, it starts with a common, precompiled base image generated using Buildroot. On top of this base, Houdini layers only the necessary test-specific components, such as particular versions of Docker, runc using an overlay. This means the same lightweight VM can be reused across multiple tests, while still ensuring that each test runs with precisely the versions of the tools it is supposed to evaluate. By isolating these versions from the host system, Houdini guarantees a controlled and repeatable testing environment regardless of changes on the host machine.

3.3 Scalability

Houdini is built to support large-scale container security evaluations without requiring extensive manual effort or scripting. While each test is executed serially to maintain isolation and determinism, the overall framework is designed to scale efficiently across hundreds or thousands of test configurations.

At the core of Houdini's scalability is its matrix-based testing model. Instead of manually defining each test case, users specify sets of options, such as Docker flags, Linux capabilities, namespace settings, or user identities that they want to explore. Houdini then automatically generates the Cartesian product of these options, producing a comprehensive list of test cases. This allows for systematic evaluation of all meaningful combinations without the need for custom scripts or duplicated logic.

Houdini's internal scheduler ensures that each test is executed in isolation within a clean virtual machine. It manages the lifecycle of each VM, booting, running the trick, capturing the output, and tearing down the environment, without user intervention. This automation allows large batches of tests to be run back-to-back,

making it possible to evaluate broad security surfaces such as how different container configurations behave across Docker and runc versions, or how changes to namespace settings influence process visibility.

To handle high test volumes efficiently, Houdini optimizes performance through the reuse of lightweight Buildroot-based VM images and precompiled runtime binaries. It avoids rebuilding environments from scratch and instead layers test-specific components on top of a shared base image using an overlay filesystem. This not only saves time but also reduces system resource usage.

3.4 Host Isolation

Houdini is explicitly designed to safely execute potentially dangerous or privilege-escalating container workloads without risking the security or stability of the host system. This is achieved through strict isolation enforced by running all tests inside QEMU-based virtual machines (VMs). This approach ensures strong separation between the test environment and the host, providing both safety and reproducibility.

Each trick is executed within its own QEMU VM, which serves as a hardware-level isolation boundary. Unlike approaches that rely solely on Linux container isolation, this VM-level sandbox guarantees that even if a container successfully escapes its sandbox, it remains confined to the guest environment. No test payload can reach or affect the host operating system, making Houdini suitable for executing untrusted or potentially malicious code safely.

Houdini further avoids any direct linkage between the host and the guest environment. No host directories are mounted into the guest, and no namespaces are shared between the host and VM. Containers are launched entirely within the guest's user, mount, PID, and network namespaces. As a result, containers inside the VM have no visibility into host processes, filesystems, or resources. Flags such as `-pid=host` only

apply within the VM context and never expose the physical host-level information.

Houdini's reliance on QEMU offers several key advantages over alternative virtualization platforms such as VMware or VirtualBox. First, QEMU supports both full system emulation and hardware-assisted virtualization through KVM, allowing it to run unmodified Linux kernels and minimalist userland environments generated by Buildroot. It can boot directly from a raw kernel image (e.g., `bzImage`) without requiring a bootloader or full OS, enabling precise control over the testing environment.

Second, QEMU supports complete command-line configuration, including CPU model, memory size, disk images, network interfaces, and serial ports. This makes it ideal for automated and reproducible test workflows. In contrast, VMware and VirtualBox rely on graphical interfaces and more static configurations, which hinder automation and scripting.

Third, QEMU is architecture-agnostic. It supports x86, ARM, MIPS, RISC-V, and other architectures, allowing for cross-platform testing. While Houdini currently focuses on x86_64, the use of QEMU ensures that it can easily be extended to other architectures in the future. VMware and VirtualBox are limited to x86-based virtualization and do not support full system emulation for other CPU architectures.

3.5 Transparency and Debuggability

Houdini is designed not only to execute tests reliably, but also to make results easy to interpret and debug. To support transparency and effective diagnosis, the framework implements structured logging for every test.

Each test execution generates a structured log file that includes the container's standard output, standard error, exit code, and a timestamped record of the configuration parameters used. These logs are stored in a standardized format and organized

by test ID, making it straightforward to trace the provenance of any result. Whether a trick passed, failed, or encountered an unexpected runtime error, the corresponding log provides clear visibility into what occurred during execution.

Houdini also tags every test result with its full configuration metadata. This includes the container runtime version, kernel version, Docker flags, capabilities, namespace modes, and any AppArmor or seccomp profiles in effect. These tags are recorded alongside the test result, enabling automated analysis, filtering, and regression tracking. For example, a user can easily compare how the same trick behaves under different runc versions or with different namespace settings, based on the metadata attached to each result.

3.5.1 Other Design Goals

In order to support the use cases mentioned above, we designed Houdini with the following goals:

1. *Reproducible Results.* A key goal of Houdini is to ensure that tests yield consistent and repeatable results. By controlling system variables and maintaining a structured testing environment, Houdini allows researchers and practitioners to compare results across different system configurations and security policies.

2. *Separation from the Host.* Houdini is designed to run tests inside a controlled containerized environment within a virtual machine (VM), ensuring that even if a trick triggers a security vulnerability, it does not compromise the host system running the tests. This design choice enhances the safety of security evaluations, preventing unintended side effects on the underlying infrastructure.

3. *Test Case Expressiveness.* Houdini tricks should be maximally expressive, such that some combination of steps can be used to achieve and test any desired result. It should be possible to define a new Houdini trick and modify existing

tricks without modifying the Houdini binary. Moreover, it should be clear from reading a defined trick precisely what steps are involved, the consequences of each step passing or failing, and the overall nature of the exploit being tested.

4. *Focus on Observing Failures, Not Preventing Them.* Houdini is built to identify security weaknesses rather than defend against attacks.

Chapter 4

Implementation

4.1 Trick Structure Overview

Houdini supports two primary modes for defining and executing tricks. The first is a YAML-based approach where a single trick is fully specified through a standalone configuration file. This method is ideal for running individual tests with carefully controlled settings, making it well-suited for targeted investigations or manual debugging. The second mode is a Python-based approach that allows users to define parameter sets as lists, which Houdini expands into a Cartesian product to form a complete test matrix. For example, parameters can be defined in Python, and Houdini will automatically generate all combinations to be tested.

4.2 YAML-Based Trick Definition

Each trick in Houdini is composed of three modular components that work together to define and execute a container-based security test: a configuration file, a trick script, and a Dockerfile. The configuration file is a YAML document that specifies the parameters for launching the Docker container, including security-related settings such as Linux capabilities, namespace modes, AppArmor profiles, user privileges, and

resource limits. These parameters are parsed by Houdini and translated directly into Docker API calls, ensuring that containers are created with precise, repeatable configurations.

The second component is the trick script, which contains the logic of the test and is executed inside the container. Typically written in Python or Bash, this script attempts a specific action—such as reading a sensitive file, sending a signal to a process, or escalating privileges—and reports the outcome by printing a success or failure message. The semantics of what constitutes a pass or fail are entirely defined by the script, giving users full control over evaluation criteria.

The final component is the Dockerfile, which builds the container image and defines the runtime environment. It includes instructions for copying the trick script into the container, installing any necessary dependencies, and specifying the base image to use. This modular architecture allows each component to be modified independently.

This YAML-based approach is ideal for running single, isolated experiments with controlled configurations, making it especially useful for debugging, demonstration, or documentation purposes.

4.3 Configuration File

Each Houdini trick begins with a configuration file, written in YAML styled format, that defines the runtime parameters used to launch the container. This file acts as the source of truth for how the container is constructed and executed. It specifies a range of Docker options including Linux capabilities (via `cap_add` and `cap_drop`), namespace settings (such as `pid_mode` and `userns_mode`), resource limits (e.g., `cpu_shares`, `mem_limit`), volume mounts, user identity, and security options such as AppArmor profiles or seccomp settings.

The configuration is declarative and follows a structured YAML schema that maps

directly to Docker's low-level run options. For example, if a user wants to disable privilege escalation, enable a read-only filesystem, or run the container as a non-root user with dropped capabilities, these settings can be encoded clearly in the configuration file. This abstraction allows users to focus on the security scenario being tested without needing to handcraft long and error-prone Docker command-line arguments.

When Houdini executes a test, the CLI parses the YAML configuration and validates it before test execution begins. It checks for required fields, ensures that runtime versions and trick paths are valid, and performs consistency checks to catch errors like incompatible or conflicting flags. Once validated, the configuration is sent to the virtual machine through the host's Flask server, where it is interpreted by the trick listener and translated into a Docker run command within the guest environment.

This structured approach ensures consistency and transparency. Since all configurations are encoded in a machine-readable format and logged alongside test results, they can be reproduced exactly in future runs or used as part of larger matrix-based experiments. It also supports modularity: the same configuration file can be reused across different tricks, or slightly modified to explore alternative isolation boundaries.

Below is an example of a configuration file used in Houdini. It defines a container with default Docker isolation settings and specifies the Dockerfile and trick path:

Figure 4.1: Configuration file for a default Docker out-of-the-box security setting.

```
name: out-of-the-box docker configuration
dockerfile:
  - path: Dockerfile
dependencies:
  - server: False
docker_config:
  - network_mode: bridge
  - read_only: False
  - security_opt: []
  - pid_mode: null
  - cpu_shares: 1024
  - volumes : []
  - mem_limit: null
  - cpuset_cpus: null
  - cpu_quota: null
  - cpu_period: null
  - cap_add: []
  - cap_drop: []
  - privileged: False
  - user: root
  - pids_limit: null
  - ipc_mode: private
  - userns_mode: null
trick:
  - path: /houdini/tricks/OutOfTheBoxExample
```

4.4 Trick Script

The trick script is the core component that defines the actual security test logic to be executed inside the container. This script is typically written in a high-level language such as Python or Bash and is copied into the container during the build process specified by the Dockerfile. Once the container is launched with the desired configuration, the trick script runs automatically and attempts a specific action, such as reading a sensitive file, accessing a kernel interface, or interacting with another

process. Its purpose is to evaluate whether the container configuration enforces the intended security boundaries.

Houdini does not impose restrictions on the scripting language used, though Python and Bash are most commonly employed due to their simplicity and compatibility with minimal container environments. The scripting interface is straightforward: the script performs a predefined action and prints a result indicating whether the trick passed or failed. These results are captured via standard output by the trick listener running inside the guest virtual machine. The listener uses these outputs, along with exit codes and standard error messages, to classify the outcome of the test.

The evaluation criteria are entirely defined within the trick script, giving users full control over what constitutes success or failure. This flexibility allows Houdini to support a wide range of test types, from low-level privilege checks to higher-level behavioral verifications. Scripts can also emit additional diagnostic messages to aid in debugging or deeper analysis.

Below is an example of a simple Python trick script that tests whether the container is able to read the contents of the `/etc/shadow` file. Under standard Docker security settings, access to this file should be restricted. If the script is able to read it, the trick is considered to have passed—indicating a misconfiguration or insufficient isolation. Otherwise, it fails.

Figure 4.2: Example trick implementation that checks for read access to `/etc/shadow`.

```
import os

def check_shadow_access():
    path = "/etc/shadow"
    if os.path.exists(path):
        try:
            with open(path, 'r') as f:
                print("Able to read /etc/shadow:")
                print("Trick passed")
        except PermissionError:
            print("Permission denied to read /etc/shadow.")
            print("Trick failed")
        except Exception as e:
            print(f"Unexpected error: {e}")
            print("Trick failed")
    else:
        print("/etc/shadow does not exist.")
        print("Trick failed")

check_shadow_access()
```

This example demonstrates how a trick can evaluate a specific security assumption—file access control in this case—and produce a deterministic result that Houdini can log and interpret. More advanced tricks may incorporate conditionals, environment checks, or IPC mechanisms, but the basic structure remains the same: run a check and report a pass/fail outcome in a clear and reproducible manner.

4.5 Dockerfile

The Dockerfile is responsible for setting up the container's runtime environment so that the trick script can execute reliably and with all necessary dependencies. It defines how the container image is built, including the selection of a base image, the installation of packages or libraries required by the trick script, and the inclusion of

the trick file itself.

Tricks in Houdini can be written in a variety of languages such as Python or Bash, so the Dockerfile must ensure that the corresponding runtime (e.g., the Python interpreter or Bash shell) is available inside the container. This often involves installing system packages using the package manager provided by the base image—for example, using `apt` in Debian-based images or `apk` in Alpine.

Another essential part of the Dockerfile is copying the trick script into the container's filesystem. This step ensures that the logic defined by the user is available at a known path when the container starts. Houdini standardizes this by copying the trick to a predefined directory (e.g., `/houdini/trick.py`) so that all containers follow the same execution convention.

The base image is selected based on the needs of the trick. Lightweight images like Alpine Linux are often preferred for simple tests because of their small size and fast startup time. However, more feature-rich images like Debian or Python-based official images may be used when additional dependencies or language runtimes are required.

Finally, the Dockerfile specifies an entrypoint or `CMD` directive to automatically run the trick script when the container launches. This ensures that the trick executes without requiring user intervention or manual commands. The entrypoint is usually a direct call to the interpreter (e.g., `python3 /houdini/trick.py`) or a shell script that wraps additional logic.

The example below demonstrates a simple Dockerfile used in Houdini to run a Python-based trick. It uses the official Python image, copies the trick into the container, and sets the default command to execute the script:

Figure 4.3: Example Dockerfile used to prepare the container environment for a Python-based trick.

```
FROM python:3.9-slim

WORKDIR /houdini

COPY trick.py /houdini/trick.py

CMD ["python3", "/houdini/trick.py"]
```

This Dockerfile is minimal yet sufficient for running Python scripts inside a container. It illustrates the typical pattern used in Houdini to prepare containers in a consistent and reproducible manner. More complex tricks may extend this file to install additional packages, configure environment variables, or mount external resources, but the core structure remains largely the same.

4.6 Python-Based Parameter Expansion Mode

In addition to supporting single-trick YAML definitions, Houdini provides a Python-based mechanism to define a matrix of configurations by specifying parameters as lists. Houdini automatically expands these parameters into a Cartesian product and generates a full suite of test cases. This approach is powerful for scaling up experiments, allowing researchers to explore the interaction of various container runtime settings systematically.

Listing 4.1: Trick parameter sets defined in Python

```
pid_mode = [None, "--pid=host"]
cap_kill = [False, True]
cap_sys_ptrace = [False, True]
privileged_opts = [False, True]
```

```
apparmor_profiles = ["apparmor=unconfined", "apparmor=docker-default
    "]
user = ["huzi"]
runtime = ["crun"]
```

Let $\mathcal{P}$ denote the parameter space. We define:

$$\mathcal{P}_{\text{pid}} = \{\varnothing, \texttt{"-pid=host"}\}$$
$$\mathcal{P}_{\text{kill}} = \{\texttt{False}, \texttt{True}\}$$
$$\mathcal{P}_{\text{ptrace}} = \{\texttt{False}, \texttt{True}\}$$
$$\mathcal{P}_{\text{priv}} = \{\texttt{False}, \texttt{True}\}$$
$$\mathcal{P}_{\text{apparmor}} = \{\texttt{"apparmor=unconfined"}, \texttt{"apparmor=docker-default"}\}$$
$$\mathcal{P}_{\text{user}} = \{\texttt{"huzi"}\}$$
$$\mathcal{P}_{\text{runtime}} = \{\texttt{"crun"}\}$$

The full parameter space is their Cartesian product:

$$\mathcal{P} = \mathcal{P}_{\text{pid}} \times \mathcal{P}_{\text{kill}} \times \mathcal{P}_{\text{ptrace}} \times \mathcal{P}_{\text{priv}} \times \mathcal{P}_{\text{apparmor}} \times \mathcal{P}_{\text{user}} \times \mathcal{P}_{\text{runtime}}$$

Each tuple $\vec{p} \in \mathcal{P}$ corresponds to a single command invocation:

$$\texttt{docker run -i } + \text{flags}(\vec{p}) + \texttt{ \$IMAGE}$$

Since $|\mathcal{P}_{\text{pid}}| = |\mathcal{P}_{\text{kill}}| = |\mathcal{P}_{\text{ptrace}}| = |\mathcal{P}_{\text{priv}}| = |\mathcal{P}_{\text{apparmor}}| = 2$ and $|\mathcal{P}_{\text{user}}| = |\mathcal{P}_{\text{runtime}}| = 1$, the total number of configurations is:

$$|\mathcal{P}| = 2^5 = 32$$

Below are the 32 commands generated by iterating over all elements in $\mathcal{P}$:

Listing 4.2: Generated Docker run commands from Cartesian expansion

```
1  docker run -i --user huzi --security-opt apparmor=unconfined --runtime crun $IMAGE
2  docker run -i --user huzi --security-opt apparmor=unconfined --cap-add CAP_KILL --runtime crun $IMAGE
3  docker run -i --user huzi --security-opt apparmor=unconfined --cap-add CAP_SYS_PTRACE --runtime crun $IMAGE
4  docker run -i --user huzi --security-opt apparmor=unconfined --cap-add CAP_KILL --cap-add CAP_SYS_PTRACE --
       runtime crun $IMAGE
5  docker run -i --user huzi --security-opt apparmor=unconfined --privileged --runtime crun $IMAGE
6  docker run -i --user huzi --security-opt apparmor=unconfined --privileged --cap-add CAP_KILL --runtime crun
       $IMAGE
7  docker run -i --user huzi --security-opt apparmor=unconfined --privileged --cap-add CAP_SYS_PTRACE --runtime crun
        $IMAGE
8  docker run -i --user huzi --security-opt apparmor=unconfined --privileged --cap-add CAP_KILL --cap-add
       CAP_SYS_PTRACE --runtime crun $IMAGE
9  docker run -i --user huzi --security-opt apparmor=docker-default --runtime crun $IMAGE
10 docker run -i --user huzi --security-opt apparmor=docker-default --cap-add CAP_KILL --runtime crun $IMAGE
11 docker run -i --user huzi --security-opt apparmor=docker-default --cap-add CAP_SYS_PTRACE --runtime crun $IMAGE
12 docker run -i --user huzi --security-opt apparmor=docker-default --cap-add CAP_KILL --cap-add CAP_SYS_PTRACE --
       runtime crun $IMAGE
13 docker run -i --user huzi --security-opt apparmor=docker-default --privileged --runtime crun $IMAGE
14 docker run -i --user huzi --security-opt apparmor=docker-default --privileged --cap-add CAP_KILL --runtime crun
       $IMAGE
15 docker run -i --user huzi --security-opt apparmor=docker-default --privileged --cap-add CAP_SYS_PTRACE --runtime
       crun $IMAGE
16 docker run -i --user huzi --security-opt apparmor=docker-default --privileged --cap-add CAP_KILL --cap-add
       CAP_SYS_PTRACE --runtime crun $IMAGE
17 docker run -i --user huzi --pid=host --security-opt apparmor=unconfined --runtime crun $IMAGE
18 docker run -i --user huzi --pid=host --security-opt apparmor=unconfined --cap-add CAP_KILL --runtime crun $IMAGE
19 docker run -i --user huzi --pid=host --security-opt apparmor=unconfined --cap-add CAP_SYS_PTRACE --runtime crun
       $IMAGE
20 docker run -i --user huzi --pid=host --security-opt apparmor=unconfined --cap-add CAP_KILL --cap-add
       CAP_SYS_PTRACE --runtime crun $IMAGE
21 docker run -i --user huzi --pid=host --security-opt apparmor=unconfined --privileged --runtime crun $IMAGE
22 docker run -i --user huzi --pid=host --security-opt apparmor=unconfined --privileged --cap-add CAP_KILL --runtime
        crun $IMAGE
23 docker run -i --user huzi --pid=host --security-opt apparmor=unconfined --privileged --cap-add CAP_SYS_PTRACE --
       runtime crun $IMAGE
24 docker run -i --user huzi --pid=host --security-opt apparmor=unconfined --privileged --cap-add CAP_KILL --cap-add
        CAP_SYS_PTRACE --runtime crun $IMAGE
25 docker run -i --user huzi --pid=host --security-opt apparmor=docker-default --runtime crun $IMAGE
26 docker run -i --user huzi --pid=host --security-opt apparmor=docker-default --cap-add CAP_KILL --runtime crun
       $IMAGE
27 docker run -i --user huzi --pid=host --security-opt apparmor=docker-default --cap-add CAP_SYS_PTRACE --runtime
       crun $IMAGE
28 docker run -i --user huzi --pid=host --security-opt apparmor=docker-default --cap-add CAP_KILL --cap-add
       CAP_SYS_PTRACE --runtime crun $IMAGE
29 docker run -i --user huzi --pid=host --security-opt apparmor=docker-default --privileged --runtime crun $IMAGE
30 docker run -i --user huzi --pid=host --security-opt apparmor=docker-default --privileged --cap-add CAP_KILL --
       runtime crun $IMAGE
31 docker run -i --user huzi --pid=host --security-opt apparmor=docker-default --privileged --cap-add CAP_SYS_PTRACE
        --runtime crun $IMAGE
32 docker run -i --user huzi --pid=host --security-opt apparmor=docker-default --privileged --cap-add CAP_KILL --cap
       -add CAP_SYS_PTRACE --runtime crun $IMAGE
```

4.7 Execution Pipeline

Houdini's execution pipeline is designed to take a user-defined trick and run it in a fully controlled and reproducible environment. The process begins on the host system, where the user selects a trick and provides its associated configuration, either through a YAML file or a Python-based matrix. Houdini's command-line interface (CLI) parses this configuration, verifies its correctness, and prepares the test case for execution. Validation includes checking that required files exist, runtime versions are available in the local cache, and parameters such as capabilities and namespace modes are syntactically valid and logically consistent.

Once the configuration is validated, Houdini launches a new QEMU virtual machine using a Buildroot-generated image as the guest operating system. This virtual machine provides strong isolation, ensuring that even if a trick contains malicious logic or exploits a container escape vulnerability, the impact remains confined to the VM. During boot, the guest VM connects to a lightweight Flask server running on the host. This server acts as the communication bridge between the host and guest environments, exposing a simple HTTP API for transferring configuration data and receiving test results.

After the guest VM is online and connected, the Flask server sends the configuration for the trick to a persistent background service inside the VM known as the trick listener. This listener is responsible for setting up and executing the test within the guest. It begins by building the Docker image using the provided Dockerfile, copying the trick script into the container, and applying all specified configuration options such as capabilities, AppArmor profiles, and user identity.

Once the image is built, the container is launched with the defined Docker run settings. The trick script runs automatically inside the container, performing its test logic and writing its output to standard output and error streams. The trick listener monitors the container's execution, capturing the exit code, output, and any

diagnostic messages produced by the trick. These results are packaged together with the configuration metadata and returned to the host via the Flask server.

On the host side, Houdini stores all test results in structured logs indexed by test ID. This log includes the full configuration, trick output, container exit status, and timestamped metadata, making it easy to trace, compare, and debug results. The entire pipeline—from configuration parsing to result capture—is designed to run without manual intervention, enabling users to queue and evaluate large sets of tests in a reliable and repeatable manner.

By combining strict virtual machine isolation, automated container orchestration, and lightweight communication via Flask, Houdini provides a robust and secure pipeline for container security testing that is both scalable and transparent.

4.8 Example Trick: Shadow File Access

To demonstrate Houdini's end-to-end workflow, a simple example trick is provided that attempts to access the `/etc/shadow` file from inside a container. This trick is not an exploit, but rather a baseline check designed to verify that basic file system isolation is functioning correctly under the container's security configuration. The motivation for this test lies in the sensitivity of the `/etc/shadow` file, which stores hashed system passwords and should only be accessible to privileged processes. Under standard Docker configurations, containers should be unable to read this file unless they are overly privileged or misconfigured.

The expected behavior, when Docker is running in its default configuration, is that access to `/etc/shadow` will be denied. The container process should either encounter a permission error or be unable to open the file at all. This trick validates that Docker's default isolation mechanisms—such as user identity mapping, read-only mounts, and AppArmor policies—are effective in preventing unauthorized access to host-sensitive

files.

From a security evaluation perspective, this test serves as a sanity check. It allows users to confirm that their Houdini setup is working correctly and that the baseline container environment enforces expected isolation boundaries. If this simple trick unexpectedly passes, it indicates a serious misconfiguration that could invalidate the results of more complex security tests. As such, this example is often used as an initial test when setting up Houdini on a new host or validating a new kernel or Docker runtime version.

The full implementation of the trick script is provided in Listing 4.2, and the corresponding container configuration is shown in Listing 4.1. When executed inside a container, the script attempts to open and read `/etc/shadow`. If it succeeds, it prints "Trick passed"; otherwise, it prints "Trick failed." These results are captured and logged by Houdini as part of the standard execution pipeline. This example highlights the modularity and transparency of Houdini's trick-based testing model and illustrates how a simple action can validate critical isolation guarantees.

4.9 Flask Server Integration

The Flask server plays a central role in Houdini's architecture by acting as the primary communication bridge between the host system and the guest virtual machine. Once the QEMU-based Buildroot VM boots, it establishes a connection to this Flask server to initiate trick execution, request runtime metadata, or receive configuration files. The server runs as a lightweight HTTP service on the host, exposing a set of well-defined endpoints that orchestrate the full test workflow.

Among the core endpoints exposed by the server are `/run-trick`, `/run-command-in-vm`, and `/server-status`. The `/server-status` endpoint is used for health checking and indicates whether the Houdini server is actively running on the host. This is imple-

mented by checking for the Python process associated with the server:

Listing 4.3: Server health check endpoint

```
@app.route('/server-status', methods=['GET'])
def server_status():
    if is_service_running('/usr/bin/python3 /houdini/server.py'):
        response_data = str(True)
        return make_response(response_data, 200)
    else:
        response_data = str(False)
        return make_response(response_data, 503)
```

The `/version-status` endpoint queries the current versions of critical components such as Docker and `runc`. It executes shell commands using Python's `subprocess` module and returns a JSON-formatted response with version information:

Listing 4.4: Runtime version information endpoint

```
@app.route('/version-status', methods=['GET'])
def version_status():
    versions = get_versions()
    return make_response(jsonify(versions), 200)
```

The most critical endpoint, `/run-trick`, handles the dispatching of a specific trick into the guest VM. When a request is made to this endpoint, the server loads the corresponding YAML configuration file, parses it, and executes the full Docker test pipeline. Output is captured using an in-memory stream to allow full logging of results:

Listing 4.5: Trick execution endpoint

```
@app.route('/run-trick/<trick_folder>/<trick>', methods=['GET'])
def run_trick(trick_folder, trick):
    TRICK_PATH = f'/houdini/tricks/{trick_folder}/{trick}'
    if not os.path.exists(TRICK_PATH):
```

```
        return jsonify({'error': f'Trick {trick} not found. PATH = {
            TRICK_PATH}'}), 404

    with open(TRICK_PATH, 'r') as file:
        trick_data = yaml.safe_load(file)

    output = io.StringIO()
    with contextlib.redirect_stdout(output):
        results = parse_trick_and_run(trick_data, request.args)

    return Response(output.getvalue())
```

For debugging and exploratory testing, the `/run-command-in-vm` endpoint allows arbitrary shell commands to be executed within the VM environment. This is particularly useful for checking environment settings or container state without needing a full trick configuration:

Listing 4.6: Run shell command in VM

```
@app.route('/run-command-in-vm', methods=['GET'])
def run_command_in_vm():
    cmd = request.args.get('cmd')
    try:
        output_lines = []
        with subprocess.Popen(
            cmd, shell=True, stdout=subprocess.PIPE, stderr=
                subprocess.STDOUT, text=True
        ) as process:
            for line in process.stdout:
                output_lines.append(line)
        return jsonify({'output': ''.join(output_lines)})
    except Exception as e:
        return jsonify({'error': str(e)}), 500
```

Together, these endpoints enable seamless interaction between the host and guest systems. They allow Houdini to manage test execution in a modular, scriptable, and reproducible manner. The server abstracts complex Docker interactions and version management behind a simple HTTP API, making it easy to integrate into larger security testing workflows.

4.10 Trick Listener and Trick Execution Logic

The trick listener is a persistent background service running inside the QEMU virtual machine. Its role is to receive trick configurations sent from the host's Flask server and execute them in a controlled, containerized environment. At the heart of this execution logic lies the `parse_trick_and_run()` function, which interprets the YAML trick configuration, builds the Docker image, launches the container with specified settings, and monitors the test.

The execution begins by parsing the YAML configuration file received via HTTP. This file is expected to contain two main sections: the `docker_config` array and the `trick` dictionary. The configuration is validated to ensure all required fields exist, and the container name is extracted from the query string arguments.

The first step is to ensure any container from a previous run does not interfere. If a container with the same name is already running, it is forcefully terminated and removed:

Listing 4.7: Container cleanup

```
1 def check_if_container_is_running(container_name):
2     try:
3         existing_container = client.containers.get(container_name)
4         print(f"Container {container_name} already exists. Removing
              it.")
5         if existing_container.status == 'running':
```

```
            existing_container.kill(signal=SIGKILL)
        existing_container.remove(force=True)
    except docker.errors.NotFound:
        print(f"No existing container named {container_name} found."
            )
```

Next, the Docker image is built from the provided trick directory. This includes copying the Dockerfile and trick script into the working context. The image is tagged using the test name to ensure clear tracking:

Listing 4.8: Docker image build step

```
def build_docker_image(dockerfile_path, image_name):
    print("Creating Docker Image")
    client.images.build(path=".", tag=image_name, rm=True)
    print(f"Image '{image_name}' built successfully.")
```

Once built, the container is launched with the exact parameters specified in the configuration. These include capabilities, AppArmor, PID namespace settings, memory constraints, and more. Houdini uses the Docker SDK to pass these options directly to `client.containers.run()`:

Listing 4.9: Container launch logic

```
container = client.containers.run(
    image=image_name,
    name=container_name,
    detach=True,
    network_mode=network_mode,
    read_only=read_only,
    security_opt=security_opt,
    pid_mode=pid_mode,
    cpu_shares=cpu_shares,
    volumes=volumes,
    mem_limit=mem_limit,
```

```
    cpuset_cpus=cpuset_cpus,
    cpu_quota=cpu_quota,
    cpu_period=cpu_period,
    cap_add=cap_add,
    cap_drop=cap_drop,
    privileged=privileged,
    user=user,
    pids_limit=pids_limit,
    ipc_mode=ipc_mode,
    userns_mode=userns_mode,
    stdout=True,
    stderr=True,
    tty=True,
)
```

Houdini then monitors the execution of the container, capturing the full stdout stream using `attach()` and waiting for the process to terminate. Once the container exits, its status code is recorded to determine if the trick passed, failed, or encountered an error:

Listing 4.10: Container monitoring and result capture

```
for chunk in codecs.iterdecode(container.attach(stdout=True, stream=
    True), "utf-8"):
    sys.stdout.write(chunk)

exit_status = container.wait()
exit_code = exit_status['StatusCode']
print(f"Container exited with code: {exit_code}")
```

The results—comprising the container's output, exit code, and execution metadata—are returned back to the Flask server, which then forwards them to the user via an HTTP response. This modular pipeline allows complete decoupling between

test specification, execution, and result retrieval.

A simplified pseudocode version of `parse_trick_and_run()` is shown below for clarity:

Listing 4.11: Pseudocode for Trick Execution

```
def parse_trick_and_run(trick_data, args):
    container_name = args.get('container_name')
    check_if_container_is_running(container_name)
    build_docker_image(trick_data['trick'][0]['path'],
       container_name)
    container = run_docker_container(...all config params...)
    monitor_and_capture(container)
```

This function glues together every step in Houdini's lifecycle: it prepares the Docker image, applies the container runtime configuration, executes the trick, and forwards the result. Its simplicity belies the modular complexity and flexibility of Houdini's test infrastructure.

4.11 Version Reporting Endpoint

To ensure complete reproducibility and traceability of test results, Houdini exposes a dedicated endpoint called `/version-status`. This endpoint provides runtime versioning information about critical components involved in container execution: Docker, `runc`, and `docker-runc`. These version numbers are essential for researchers to accurately interpret results and replicate experiments across systems with different host configurations.

When a request is made to `/version-status`, the Flask server calls a utility function named `get_versions()`. This function executes shell commands using Python's `subprocess.check_output()` to retrieve version strings from each runtime binary.

It gracefully handles errors such as missing binaries or unexpected output formats, returning a JSON object with version strings or fallback messages.

The current implementation supports Docker, `runc`, and `docker-runc`, but can be extended to include kernel versions or other runtime environments. By returning this information as part of the test log, Houdini guarantees that every trick result is linked to a precise system state, supporting accurate debugging and longitudinal analysis.

Below is the code that implements this endpoint:

Listing 4.12: Version reporting logic for /version-status

```
@app.route('/version-status', methods=['GET'])
def version_status():
    versions = get_versions()
    return make_response(jsonify(versions), 200)

def get_versions():
    versions = {}

    try:
        docker_output = subprocess.check_output(["docker", "--
            version"], stderr=subprocess.STDOUT)
        versions['docker'] = docker_output.decode().strip()
    except subprocess.CalledProcessError as e:
        versions['docker'] = f"Error: {e.output.decode().strip()}"
    except FileNotFoundError:
        versions['docker'] = "Docker not found"

    try:
        runc_output = subprocess.check_output(["runc", "--version"],
             stderr=subprocess.STDOUT)
        versions['runc'] = runc_output.decode().strip()
    except subprocess.CalledProcessError as e:
```

```
21          versions['runc'] = f"Error: {e.output.decode().strip()}"
22      except FileNotFoundError:
23          versions['runc'] = "runc not found"
24
25      try:
26          docker_runc_output = subprocess.check_output(["docker-runc",
                "--version"], stderr=subprocess.STDOUT)
27          versions['docker-runc'] = docker_runc_output.decode().strip
                ()
28      except subprocess.CalledProcessError as e:
29          versions['docker-runc'] = f"Error: {e.output.decode().strip
                ()}"
30      except FileNotFoundError:
31          versions['docker-runc'] = "docker-runc not found"
32
33      return versions
```

This mechanism supports Houdini's goal of rigorous reproducibility by anchoring trick results to specific software states. Extending this logic to include kernel version detection—by invoking `uname -r`—would further improve the system's ability to document host-level conditions that influence container behavior.

4.12 Dynamic Runtime Dispatch

Houdini supports dynamic selection of container runtimes—specifically `runc` and `crun`—through both YAML-based trick definitions and Python-based test matrix expansions. This flexibility allows researchers to evaluate how different low-level runtimes handle the same security configurations, offering deeper insight into runtime-specific behavior.

Runtime selection is user-configurable in two ways. First, for YAML-defined tricks,

the `runtime` field can be set explicitly (e.g., `crun`, `runc`) in the `docker_config` section. Second, in matrix-driven experiments, runtime is one of the test parameters expanded using Cartesian product logic. A list of runtimes is defined in the configuration module and passed into the combination generator:

Listing 4.13: Runtime as a configurable test parameter in matrix mode

```
runtime = ["runc", "crun"]
...
def generate_combinations(config):
    fields = [
        ...
        config.get("runtime") or [None]
    ]
    return list(itertools.product(*fields))[:MAX_TESTS]
```

At runtime, the selected container runtime binary (e.g., `crun`) is injected into the guest virtual machine (QEMU) by copying the appropriate executable into the Buildroot overlay directory `/usr/bin` before VM boot. This is done dynamically based on the version set in `houdini_config.py`. The correct binary is fetched from a pre-extracted folder of supported runtime versions:

Listing 4.14: Injecting crun into the Buildroot overlay

```
get_crun_binaries() {
    ...
    src_crun=$(find "$extracted_folder" -type f -name "crun-*linux-amd64
    cp "$src_crun" "${overlay_bin}/crun"
    chmod +x "${overlay_bin}/crun"
}
```

The Docker run command issued inside the VM includes the `-runtime` flag if a

runtime is specified. This enables Houdini to test container behavior across multiple OCI runtimes within a uniform framework:

Listing 4.15: Runtime applied during container launch

```
if isinstance(runtime, str):
    cmd += ["--runtime", str(runtime)]
```

This modular dispatch mechanism is designed for future extensibility. Support for additional runtimes such as `youki`, `Kata Containers`, or `gVisor` can be added by extending the binary injection logic and updating the configuration schema to recognize new runtime identifiers.

By making the container runtime a first-class configurable dimension in experiments, Houdini enables nuanced differential testing that can expose inconsistencies, vulnerabilities, or behavioral discrepancies across runtime implementations.

4.13 Trick Parameter Expansion Logic

Houdini supports large-scale, automated testing through its ability to expand parameterized test configurations into a full matrix of trick executions. This is achieved by defining each configuration parameter as a list—such as `pid_mode`, `cap_sys_ptrace`, or `user`—and computing the Cartesian product across all parameters. Each unique combination forms one specific instance of a trick to be tested.

Internally, Houdini loads these parameters from Python modules corresponding to the selected trick. Using Python's `importlib`, it dynamically loads the values from each variable into a unified configuration dictionary. Parameters not explicitly defined default to a single-item list with a `None` value to ensure uniformity during expansion.

To generate the matrix, Houdini uses Python's `itertools.product()` to compute the Cartesian product across all lists. This approach is simple, efficient, and scales

well to a moderate number of parameters. Each combination is then passed into a container execution function that builds the necessary Docker run command for that configuration.

To limit runtime and avoid combinatorial explosion, Houdini caps the number of generated tests using the `MAX_TESTS` constant, which discards excess combinations if the matrix size becomes too large. This trade-off maintains a balance between thoroughness and execution feasibility.

Below is a simplified pseudocode version of the expansion and execution logic, taken from Houdini's core driver script:

Listing 4.16: Generating and iterating over parameter combinations

```
def load_config(trick_name):
    config_module = importlib.import_module(f"{trick_name}")
    config = {}
    for var in ["pid_mode", "cap_kill", "cap_sys_ptrace",
                "privileged_opts", "apparmor_profiles", "user",
                "net_bind_service", "cap_drop_all", "runtime"]:
        config[var] = getattr(config_module, var, [None])
    return config

def generate_combinations(config):
    fields = [
        config.get("pid_mode"),
        config.get("cap_kill"),
        config.get("cap_sys_ptrace"),
        config.get("privileged_opts"),
        config.get("apparmor_profiles"),
        config.get("user"),
        config.get("net_bind_service"),
        config.get("cap_drop_all"),
        config.get("runtime")
```

```
    ]
    return list(itertools.product(*fields))[:MAX_TESTS]
```

This design enables Houdini to efficiently run comprehensive test suites with varying parameters across container runtimes, user identities, namespace modes, and Linux capabilities. It also supports extensibility, as new parameters can be added to the configuration without altering the underlying expansion logic.

4.14 Security Considerations in Host-Level Code

Houdini is designed to execute potentially dangerous container configurations and scripts in a safe and reproducible environment. To ensure host system integrity, Houdini carefully separates test execution from the host by sandboxing all container activity within a QEMU virtual machine. This isolation guarantees that even if a containerized trick exploits a misconfiguration or escapes the container runtime, it remains confined within the guest VM, protecting the host from compromise. For this reason, all subprocess executions—including Docker image builds, container runs, and file access tests—are offloaded to the VM environment.

This design decision also ensures that Houdini's host-side Python code does not directly interact with untrusted trick scripts or configurations. By shifting the attack surface into the VM, Houdini minimizes privilege exposure and keeps the host clean of any temporary state or Docker containers created during testing.

Another key consideration is Docker Hub's rate limiting policy. Without authentication, anonymous users are restricted to a limited number of image pulls and builds per six-hour window. To avoid encountering these rate limits during large-scale test execution, Houdini requires authenticated Docker access using a personal access token. This is handled in the host logic via a login step:

Listing 4.17: Logging into Docker Hub to bypass image build limits

```
username = os.getenv("DOCKER_USERNAME")
password = os.getenv("DOCKER_TOKEN")
client = docker.from_env()
client.login(username=username, password=password)
```

This login process ensures that authenticated API calls to Docker Hub are used when building and pulling images. It also allows users with Docker Pro or Team subscriptions to take advantage of higher rate limits and private repository access.

4.15 Virtual Machine Setup and Execution

Houdini runs all container-based security tests inside a QEMU virtual machine to ensure strong host isolation and reproducibility. Each virtual machine is spun up dynamically for every trick, booting into a minimal Linux distribution generated by Buildroot. This architecture isolates all container behavior from the host operating system and allows precise control over the kernel and filesystem.

4.15.1 Purpose of Virtualization

The use of virtualization provides several key benefits:

- **Security Isolation:** Containers cannot directly affect the host system, even if misconfigured or compromised.
- **Reproducibility:** A consistent environment is provided for every test, removing host system variability.
- **Custom Kernel Support:** Researchers can test custom kernel builds or apply experimental patches without rebooting the host.

4.15.2 QEMU Execution Command

Houdini uses QEMU to run a VM with a custom kernel and root filesystem. The Python method responsible for starting the virtual machine is shown below:

Listing 4.18: Python method to start the QEMU VM in Houdini

```
def start_vm(self, kernel=get_absolute_path(f"/kernels/{
    KERNEL_VERSION}/arch/x86/boot/bzImage"), drive=get_absolute_path(
    "/buildroot/output/images/rootfs.ext2")):
qemu_cmd = [
"qemu-system-x86_64",
"-smp", str(CPU_CORES),
"-m", str(VM_RAM),
"-kernel", kernel,
"-drive", f"file={drive},if=virtio,format=raw",
"-append", "apparmor=1 security=apparmor rootwait root=/dev/vda
    console=tty1 console=ttyS0 loglevel=0 cgroup_enable=cpuset
    cgroup_enable=memory",
"-serial", "mon:stdio",
"-net", "nic,model=virtio",
"-net", f"user,hostfwd=tcp::{PORT}-:{PORT}",
"-nographic",
"-virtfs", "local,path=/home/huzi/Desktop,mount_tag=hostshare,
    security_model=mapped-file,id=hostshare"
]
```

4.15.3 Key VM Configuration Options

The following options in the QEMU command are critical to Houdini's testing workflow:

- `-kernel`: Loads a specific Linux kernel, enabling experiments across kernel versions or with custom patches.

- `-drive if=virtio`: Uses a virtio-backed root filesystem for fast disk access and compatibility with Buildroot images.
- `-append`: Passes kernel boot parameters, including enabling AppArmor and cgroups for isolation enforcement.
- `-serial mon:stdio`: Routes guest output to the host terminal for logging and debugging.
- `-nographic`: Disables the graphical interface to simplify deployment and speed up boot time.
- `-virtfs`: Mounts a host folder into the VM using Plan 9 FS, useful for accessing logs or artifacts.
- `-net nic` and `-net user,hostfwd`: Sets up a virtual NIC and forwards a TCP port for communication between the host and guest (e.g., for Flask server requests).

4.15.4 Lifecycle of a Virtual Machine

Each trick triggers a complete VM lifecycle:

1. **Boot**: A QEMU virtual machine is started with the specified kernel and rootfs.
2. **Connect**: A Flask server on the host communicates with a Python service inside the VM.
3. **Execute**: The trick is executed inside the VM, typically via Docker, and output is captured.
4. **Shutdown**: The VM is destroyed after test execution, ensuring isolation between runs.

This tightly controlled VM-based execution model is central to Houdini's reliability, safety, and reproducibility guarantees.

Chapter 5

Evaluation

To validate Houdini's effectiveness as a container security testing framework, we designed a diverse set of evaluations that reflect its core goals: reproducibility, scalability, performance, isolation, extensibility, and robustness. These tests were conducted to ensure that Houdini performs consistently across various environments and use cases, while also remaining resilient to edge cases and user modifications.

We first assess reproducibility, a foundational principle of Houdini, by executing identical tests across machines with different hardware specifications. This verifies that test outcomes are not tied to the underlying system architecture. Additionally, we verify reproducibility over time by pinning the Docker, kernel, and runc versions and rerunning the same configurations after a delay. This demonstrates Houdini's ability to deliver stable and consistent results, even as the underlying system evolves.

To evaluate scalability, we run a large batch of tricks (up to 32) across multiple configurations in a single session. This allows us to measure runtime, memory usage, and failure rates, which offers insight into Houdini's performance under load. Complementing this, we conduct performance benchmarking by timing the full lifecycle of a trick, including VM boot, container setup, and result capture. These times are compared against manual baseline executions to quantify Houdini's overhead.

Host isolation is validated through a deliberately malicious trick designed to attempt writes to physical host-accessible paths. Success in this specific evaluation is defined by the inability of the containerized code to affect the physical host.

Next, we examine version variability support by constructing a test matrix of Docker, runc, and Linux kernel versions. Successful execution of tricks across this matrix demonstrates Houdini's compatibility and robustness in diverse runtime environments. Finally, we assess extensibility by integrating a newly developed trick that probes for a specific capability. The ease of adding and running the new test without modifying core framework logic showcases Houdini's modular design and developer-friendly architecture.

5.1 Reproducibility Across Time with Version Pinning

This experiment evaluates Houdini's ability to reliably reproduce test results over time using a version-pinned container environment. The purpose of this experiment is to determine whether Houdini can serve as a reliable testing framework when executed repeatedly over time under controlled, version-pinned conditions. Version pinning refers to explicitly locking the versions of software components, such as Docker, runc, and the Linux kernel, to prevent variation due to automatic updates, rebuilds, or environmental drift. Even slight changes in runtime binaries, system dependencies, or build environments can introduce behavioral drift that invalidates prior findings and undermines test integrity.

To test reproducibility across time, we define two time points: T_0, the initial execution, and T_1, a later re-execution under identical conditions. If Houdini produces the same outputs at both time points, reproducibility is confirmed. We hypothesize that if the trick stays the same and the software environment doesn't change, Houdini

will give the same test results every time. This would make it a reliable tool for running long-term container security tests.

Table 5.1: Virtual Machine Experimental Configuration

Parameter	**Value**
Virtualization Platform	QEMU
vCPUs	4
RAM	12 GB
Root Filesystem	ext2
Kernel Version	Linux 6.0 (custom build)

Table 5.2: Version-Pinned Components

Component	**Pinned Version**
Docker	27.0.3
runc	1.1.13
Linux Kernel	6.0

To ensure binary-level consistency between runs T_0 and T_1, we calculated the SHA256 hashes of each binary. These hashes act as a cryptographic fingerprint to confirm that the binaries remain unchanged throughout the experiment. These values were re-verified at the time of re-execution using the `sha256sum` utility to confirm that no environmental drift had occurred.

Table 5.3: Reproducibility Verification Across Time

Component	**SHA256 @ T0**	**SHA256 @ T1**
Docker	`898c...5e3`	`898c...5e3`
runc	`4799...0a1`	`4799...0a1`
Kernel	`3b03...1c7`	`3b03...1c7`

The AppArmor detection trick (see Listing 5.1) is used to check if the container is running with AppArmor security enabled, specifically under the ‘docker-default‘ profile. It’s a simple and effective way to see if Linux’s AppArmor is active inside the container. The trick works by reading the file ‘/proc/1/attr/current‘, which shows the AppArmor profile applied to the container’s main process. If the profile includes ‘docker-default‘, the trick passes, meaning AppArmor is working correctly. If the profile is missing or different, the trick fails, which could mean that AppArmor is turned off or not supported. This trick helps identify whether the container’s security setup has changed across different Docker versions, kernel versions, or system settings.

The `apparmor_check.sh` script is placed in `/usr/bin/` and included via the overlay. Its purpose is to detect whether Docker applies AppArmor confinement, which varies by Docker version. The script attempts to write to /proc/sys/kernel/randomize_va_space, a location typically blocked by the default AppArmor profile.

Listing 5.1: AppArmor Enforcement Trick Script

```
#!/bin/sh

echo 1 > /proc/sys/kernel/randomize_va_space 2>/dev/null

if [ $? -eq 0 ]; then
    echo "Trick failed: AppArmor not enforced"
else
    echo "Trick passed: AppArmor enforcement detected"
fi
```

This trick is sensitive to Docker version behavior. Older Docker versions may not apply AppArmor profiles by default, resulting in successful writes and a failed trick. In contrast, modern versions typically enforce the `docker-default` profile, causing the trick to pass.

At time T_0, the environment was constructed using the pinned versions of Docker,

runc, and the Linux kernel. The container was run with the AppArmor trick script. All binary hashes were verified against the manifest, confirming that the test environment matched the intended configuration exactly.

Listing 5.2: Running the AppArmor Trick in a Container

```
1 docker run --rm myimage /apparmor_check.sh
```

Result: Trick passed

At time T_1, after a delay, the Buildroot image was rebuilt using the same configuration and the same overlay directory. This ensured that all injected binaries were identical to those used at T_0. All hashes matched the entries from T_0, confirming no binary drift had occurred. The container was again executed using the same AppArmor trick.

Listing 5.3: Running the AppArmor Trick in a Container (T_1)

```
1 docker run --rm myimage /apparmor_check.sh
```

Result: Trick passed

This experiment shows that Houdini can consistently reproduce results over time, making it a reliable tool for long-term and repeatable security testing. By locking in the versions of all key components, checking file integrity, and using a security-sensitive trick, Houdini avoids false consistency and detects real changes when they occur. While this test used an AppArmor enforcement check, the same approach can be used for more complex security tests like checking for exploits where stable and repeatable results are essential.

We define a basic reproducibility metric R as the percentage of test cases for which the result at time T_0 matched the result at time T_1:

$$R = \frac{\#\text{ of consistent test results}}{\text{total test cases}} \times 100\%$$

For this experiment:

$$R = \frac{2}{2} \times 100\% = 100\%$$

It is important to note that some tricks may always produce the same result, even if environmental drift occurs. In those cases, the result looks consistent, but it does not mean the environment stayed the same. An example of a trick that always produces the same result even when Docker, `runc`, or the kernel changes is a simple check for the presence of a file inside the container. For instance, Listing 5.4 verifies whether a specific file exists:

In this case, the output is solely a function of the script logic and the presence or absence of a file. It depends entirely on the internal state of the container's filesystem and not on any external behavior of Docker, `runc`, or the kernel. There are no kernel-level, docker or runc mechanisms that affect the result. This deterministic behavior is a direct consequence of the scripting language itself and the semantic guarantees of its evaluation. This highlights the importance of distinguishing between true environmental consistency and tricks that are trivially reproducible due to their independence from external state(s).

Listing 5.4: Trick that is invariant to environment changes

```
#!/bin/sh
if [ -f /run/trigger_file ]; then
    echo "Trick passed"
else
    echo "Trick failed"
fi
```

5.2 Reproducibility Across Machines

A property of any empirical testing framework is the ability to produce consistent results regardless of the underlying hardware. In security testing, this reproducibility is essential for verifying claims, comparing results across research environments, and enabling long-term validation of behavior across versions. However, in practice, subtle differences in CPU architecture, I/O latency, or system timing can introduce variability. This evaluation investigates whether Houdini's virtualized testing architecture effectively abstracts away host-level differences to ensure consistent trick execution.

To evaluate this, we selected two tricks written in `Python 3`:

1. One trick checks for the presence of a file `/run/trigger_file` and prints a fixed output.

2. Another trick writes a small temporary file to disk, reads it back, and confirms its contents.

These tricks were executed on two different machines with distinct hardware:

Table 5.4: Comparison of Machine A and Machine B

Specification	**Machine A**	**Machine B**
CPU Model	Intel(R) Core(TM) Ultra 7 155H	Intel Xeon (Skylake, IBRS)
vCPUs	22	16
CPU Memory	16GB	32GB
GPU	GeForce RTX 3050	NVIDIA RTX 4070 Ti Super
GPU Memory	6 GB	16GB
Disk Space	1 TB	512GB
Operating System	Ubuntu 24.04.2 LTS	Ubuntu 22.04.5 LTS

Both systems ran the same version of Houdini, using identical Buildroot VM images, Linux kernel (v6.0), Docker version, and runc version. Also, QEMU was configured identically.

Listing 5.5: Trick #1

```
#!/usr/bin/env python3
import os

if os.path.exists("/run/trigger_file"):
    print("Trick passed")
else:
    print("Trick failed")
```

Listing 5.6: Trick #2

```
#!/usr/bin/env python3

with open("/tmp/testfile.txt", "w") as f:
    f.write("Houdini check")

with open("/tmp/testfile.txt", "r") as f:
    content = f.read()

if content == "Houdini check":
    print("Trick passed")
else:
    print("Trick failed")
```

On both machines, both tricks produced identical output and showed no variation in stdout. Houdini's internal logs containing output matched exactly across runs. For the write-read trick, the generated file was hashed inside the VM using SHA256, with all outputs matching across machines. Minor timing fluctuations ($\pm$0.01s) were observed but had no effect on correctness. These findings confirm that Houdini's virtualized execution environment abstracts away hardware variability, providing strong guarantees of reproducibility.

These results validate that Houdini's use of virtualized environments and con-

Table 5.5: Reproducibility Comparison Across Machines

Metric	Machine A	Machine B	Difference
Stdout Output	`Trick passed`	`Trick passed`	None
Exit Code	0	0	None
Wall-clock Time (avg)	0.12s	0.13s	+0.01s
CPU Utilization	4%	4%	None

sistent root filesystems allows deterministic test logic to execute identically across hosts. By isolating each trick in a self-contained VM, Houdini ensures trick behavior remains invariant, enabling reliable cross-system comparison and reproducibility. This strengthens confidence in test outcomes even when the hardware cannot be assumed.

One potential concern in cross-machine testing is that physical host-level kernel settings, such as security modules, sysctl values, or namespace policies could leak into or interfere with the VM and affect test behavior. For example, Linux's Yama security module controls ptrace restrictions via the ptrace_scope sysctl, which varies across distributions. On a typical Ubuntu host, ptrace_scope may default to 1 (restricted), while hardened environments might use 2 or 3, disallowing debugging even with capabilities.

However, Houdini's design explicitly mitigates this threat. All tests are executed inside a QEMU VM that runs its own Linux kernel, fully isolated from the host. Host kernel modules like Yama are not visible or inherited by the guest kernel. This means even if Machine A has ptrace_scope=3 and Machine B uses the default 1, the VM loaded into Machine A will behave identically as the VM in Machine B. Houdini controls the kernel config and root filesystem in the guest environment, ensuring uniform policy enforcement across machines.

More generally, no physical host-level AppArmor profiles, SELinux settings, or LSM configurations apply inside the VM. Even sysctl values, such as those affecting networking, file descriptor limits, or namespaces are isolated to the VM's kernel and

cannot be influenced by the physical host. As a result, machine-specific hardening policies do not affect the determinism of trick execution.

This design choice is central to Houdini's reproducibility goal: by treating the guest VM as a sealed, deterministic black box, it ensures that only explicitly configured inputs like Docker version, kernel version, runc version, and trick code affects outcome.

5.3 Host Isolation Validation

This experiment is designed as an informal security evaluation rather than a rigorous formal verification. Its goal is not to prove beyond all possible doubt that VM escape vulnerabilities are absent. Instead, the aim is to provide practical evidence that, under typical conditions, Houdini's architecture does not expose any clear or unintended paths through which a containerized workload could influence or compromise the physical host.

To assess this, we use a controlled test: a container attempts to write to a file path that would typically point to a host-mounted directory, if such sharing were accidentally enabled. This simulated path acts as a canary, alerting us to any misconfigurations or unexpected leakages, such as shared folders or improperly scoped namespaces, that could break the intended isolation. If the container is able to write to this location, it would indicate a breach in the containment model.

While this approach cannot detect deeper issues such as hypervisor vulnerabilities, low-level device exploits, or side-channel attacks, it is well-suited for detecting more common and operational misconfigurations. In real-world scenarios, these kinds of setup errors are often the cause of unintended host exposure. Thus, this method offers a practical way to confirm that Houdini's default setup maintains strong separation between the host and guest environments.

A full formal security analysis, including techniques like fuzzing, symbolic execution, or hypervisor introspection would provide deeper coverage but is out of scope for this specific evaluation.

We designed a trick (see Listing 5.13) to evaluate whether a container running inside Houdini's VM can access the physical host system. This test simulates an escape attempt by trying to write to a file path that would typically point to a shared folder if one were configured.

Listing 5.7: Escape Trick: Simulated Write to Host-Mounted Path

```
#!/bin/sh
# Attempt to write to a path that would be host-mounted if shared
    folders were enabled
TOUCH_PATH="/mnt/host_root/hacked.txt"

echo "Attempting to write to possible host mount at $TOUCH_PATH"
echo "HACKED" > "$TOUCH_PATH" 2>/dev/null

# Try to detect if the write succeeded
if [ -f "$TOUCH_PATH" ]; then
    echo "Trick passed: Possible host write detected!"
else
    echo "Trick failed: No host write access."
fi
```

As always, the container was executed inside a QEMU VM that does not expose any host directories and uses no shared folders. The container was started using the following command:

Listing 5.8: Escape Trick: Simulated Write to Host-Mounted Path

```
docker run --rm alpine /host_escape_trick.sh
```

Output:

Listing 5.9: Escape Trick: Simulated Write to Host-Mounted Path

```
Attempting to write to possible host mount at /mnt/host_root/hacked.
    txt
Trick failed: No host write access.
```

No file named `hacked.txt` appeared on the physical host, confirming that the simulated attack was fully contained within the VM.

While shared folders are a common method for transferring files between a VM and its physical host, their use inherently weakens the isolation boundary. If a container within the VM has access to a host-mounted directory, it may write or overwrite files that become visible outside the VM. This creates a deliberate escape route that must be managed with caution.

However, Houdini avoids this problem entirely by not relying on shared folders for interprocess communication or result reporting of tricks. As mentioned prior, this is because Houdini uses a lightweight Flask-based HTTP server running on the host. When a trick completes execution inside the VM, it sends its result, such as pass/fail status or diagnostic output back to the host via an HTTP POST request over a user-mode QEMU network port.

This design has several advantages:

- It eliminates the need to mount any part of the host filesystem inside the VM.
- It decouples file I/O from test reporting, reducing the attack surface.
- It enables fine-grained control over what data leaves the VM and how it is structured.

This experiment confirms that Houdini provides strong host isolation by default. Malicious trick code that attempts to write to host-accessible paths is safely confined within the VM. The inclusion of a clearly defined threat model ensures that the assumptions and limits of containment are well-scoped. Furthermore, the use of

a Flask-based communication channel, rather than shared folders, enhances isolation by avoiding any direct file-based interaction between the guest and host. Overall, Houdini's VM-based architecture combined with controlled, network-based result reporting offers a secure foundation for container security testing.

5.3.1 Threat Model

The threat model for this evaluation assumes that the attacker has full control over arbitrary code executed inside a container running within the VM. This reflects a realistic scenario where untrusted or potentially malicious container workloads are tested to evaluate security boundaries. The attacker may attempt to exploit kernel vulnerabilities, access unintended resources, or perform file system operations that target the host. Typical actions include attempting to write to host-mounted directories, reading sensitive data, or interacting with other host processes.

However, the attacker's capabilities are strictly confined to the VM. They have no access to the physical host machine, nor are they granted elevated privileges beyond what is available inside the containerized guest. The container may share namespaces with the host, and the kernel it interacts with is the one booted inside the VM, not the host kernel.

However, in Houdini's design, these namespace-sharing configurations are restricted to the context of the VM only. That is, even if a container is launched with `-pid=host`, it shares the PID namespace only with the VM, not the physical host. Likewise, shared network or IPC namespaces remain confined to the QEMU instance. This ensures that while Houdini supports evaluation of shared-namespace behavior, any unintended access or compromise remains sandboxed inside the VM. The attacker cannot affect the physical host's kernel, and cannot persist beyond the lifecycle of the VM.

The goal of this threat model is to validate that Houdini can safely execute such

container scenarios without compromising the integrity of the physical host. Even if the attacker succeeds in exploiting the container or VM environment, the outer boundary provided by QEMU must prevent any direct impact on the physical host.

5.3.2 Baseline Comparison: Without VM Isolation

To better understand the value of Houdini's VM-based architecture, it is useful to compare it to traditional container deployments that run directly on the physical host. In such environments, containers share the host's kernel and rely on namespace and cgroup isolation to enforce security boundaries. While these mechanisms are generally effective, they are not foolproof, especially when advanced Docker features or misconfigurations come into play.

Flags like `-pid=host`, `-net=host`, and `-ipc=host` allow containers to join the host's namespaces. When used, these flags remove key layers of isolation by allowing the container to see or interact with physical host-level (in this case) resources such as process IDs, network interfaces, or shared memory segments. These options are sometimes enabled for debugging, performance, or compatibility reasons but significantly expand the container's visibility and control over the host environment.

Even more critically, the use of volume mounts (e.g., `-v /:/host`) can give a container direct access to the host filesystem. If write access is granted intentionally or through misconfiguration, an attacker could overwrite sensitive files such as `/etc/passwd`, replace binaries, or inject persistent malware. In practice, many container escapes and privilege escalations have stemmed from overly permissive volume mounts or the misuse of the `-privileged` flag, which grants broad access to kernel interfaces and devices.

In contrast, Houdini enforces a strict boundary between the containerized workload and the physical host by placing containers inside a disposable VM. Even if a container is launched with aggressive options like `-privileged` or `-pid=host`, these

privileges are scoped only to the VM, not the physical host. This means that even misconfigured or malicious containers cannot directly interact with the physical hosts kernel, devices, or filesystem.

Additionally, Houdini's architecture ensures that test environments are short lasting. Each container runs inside a fresh VM instance with no persistence beyond the test lifecycle. This further reduces risk by eliminating opportunities for long-term compromise or lateral movement.

Ultimately, this baseline comparison shows that Houdini provides a stronger isolation guarantee than conventional Docker deployments. By decoupling the guest environment from the host through virtualization, Houdini limits the potential for container exploits and enforces a security boundary that is significantly harder to breach.

5.3.3 Consideration of VM Escape Attacks

VMs are widely regarded as strong isolation boundaries between guest and host systems. However, they are not immune to attack. A VM escape occurs when code executing inside the guest environment breaks out of its confinement and gains unauthorized access to the host. These attacks typically target vulnerabilities in the hypervisor, emulated device interfaces, or low-level system components.

Several high-profile vulnerabilities demonstrate the feasibility of VM escapes. For example, CVE-2015-3456 (VENOM) exploited a flaw in QEMU's virtual floppy disk controller to achieve arbitrary code execution on the host [2]. Similarly, speculative execution vulnerabilities such as CVE-2017-5715 (Spectre) [3] and CVE-2019-11135 (ZombieLoad) [4] enabled attackers to leak host memory data from within a guest. These examples, though rare and usually mitigated through timely patches, underscore the importance of minimizing the attack surface of the virtualization layer.

Houdini is not designed to test the security of QEMU or the underlying hypervisor.

However, its architecture incorporates multiple safeguards that reduce the risk and impact of VM escape attempts. First, Houdini uses QEMU in user-mode without access to host hardware, kernel modules, or direct device passthrough. Second, the guest OS is a minimal Buildroot-based system, which avoids unnecessary services, drivers, and user-facing software that could be exploited from within the container. Third, common high-risk configurations such as shared folders, USB passthrough, or dynamic device mounting are explicitly avoided.

While no virtualized system is immune to escape-class vulnerabilities, Houdini's design adheres to best practices for secure VM isolation. Its architecture limits exposure to known escape vectors, confines all container execution to a disposable environment, and avoids risky host-guest integration mechanisms. These decisions ensure that even in the presence of an unknown or unpatched vulnerability, the practical risk to the physical host remains minimal.

5.4 Extensibility Evaluation

Here, we evaluate Houdini's extensibility by introducing a new custom trick to demonstrate that no modifications to the core orchestration logic is required. The goal is to confirm that new security scenarios can be added through configuration alone, and with the goal of highlighting Houdini's modular and scalable testing capabilities.

To implement the new trick, two components were created: (1) a configuration file defining the test matrix, and (2) a trick script that contains the logic executed inside the container.

A new configuration file was placed in the `trick_configs/` directory. This file specifies the test parameter space, such as Docker runtime flags and container-level security settings. The following variables were defined:

Listing 5.10: Trick Configuration Parameters

```
pid_mode = [None, "--pid=host"]
cap_kill = [False, True]
cap_sys_ptrace = [False, True]
privileged_opts = [False, True]
apparmor_profiles = ["apparmor=unconfined", "apparmor=docker-default
    "]
user = ["huzi"]
```

Each variable represents a dimension in the test matrix:

- **pid_mode**: Whether to isolate or share the PID namespace with the host.
- **cap_kill**: Whether to include the `CAP_KILL` capability.
- **cap_sys_ptrace**: Whether to include the `CAP_SYS_PTRACE` capability.
- **privileged_opts**: Whether the container runs in `-privileged` mode.
- **apparmor_profiles**: Specifies the AppArmor profile to use.
- **user**: Defines the user identity inside the container.

In addition to the configuration, a corresponding trick script must be placed in the `tricks/` directory inside the overlay filesystem. This file contains the actual logic to be executed inside the container. For example, a script might attempt to send a signal to a host process or inspect the contents of a protected namespace. In our evaluation, we created a simple trick that checks if a file exists in a container. The script is named to match the configuration and is automatically included in the container image during build time.

Listing 5.11: Trick Configuration Parameters

```
#!/bin/sh

TARGET_FILE="/run/trigger_file"
```

```
4
5  echo "Checking for existence of $TARGET_FILE"
6
7  if [ -f "$TARGET_FILE" ]; then
8      echo "Trick passed: File exists."
9  else
10     echo "Trick failed: File not found."
11 fi
```

By separating trick logic from orchestration, Houdini allows new security tests to be created without modifying the framework itself. As long as a valid configuration file and corresponding trick script are provided, Houdini's pipeline takes care of the rest.

Execution Flow

Once the new configuration and trick script are in place, the user activates the trick by setting the `TRICK` variable in `constants/houdini_config.py` (e.g., `TRICK = "file_exists"`).

Listing 5.12: Trick Configuration Parameters

```
1  TRICK = "FileExists"
```

Generalizability

The success of this new trick demonstrates more than just a single functional test, it confirms the correctness and generality of Houdini's extensibility model. This is because Houdini treats all tricks as interchangeable components, executed through the same standardized interface. Once a new trick is defined and referenced through a configuration file, Houdini automatically performs the following actions regardless of the trick's internal logic:

- Loads the associated configuration without requiring logic changes.
- Injects the corresponding trick script into the container image during build.
- Executes the container with the correct flags inside an isolated VM.
- Captures trick output through a uniform logging mechanism.

There is no tight coupling between the orchestration logic and the contents of the trick. The system does not assume or validate the purpose of the script, it simply executes it. Therefore, if one trick can be successfully added, executed, and logged using this path, any trick that follows the same structure can be integrated as well.

This property demonstrates that Houdini's design is not tailored to specific tests but is built around a flexible, configuration-driven framework. As long as future tricks conform to the same simple interface, they can be added without modifying the system, validating that the extensibility mechanism is generalizable.

Conclusion

This workflow demonstrates Houdini's modular architecture. Test logic (the trick file) and configuration are fully decoupled from orchestration logic. New scenarios can be added with minimal effort, requiring only a trick script and a config file. The core system remains unchanged, which makes Houdini highly extensible and suitable for evaluating new runtime behaviors, emerging vulnerabilities, or evolving isolation mechanisms in a scalable and automated fashion.

5.5 Version Variability Support

Container runtime behavior evolves significantly over time. Docker, runc, and the Linux kernel introduce new APIs, deprecate flags, and change default security configurations across versions. To remain effective, a security test framework like Houdini

must support this variability without requiring manual adaptation. This evaluation asks: Can Houdini reliably build and execute security tests across historical and modern container ecosystems with no changes to its internal logic?

To answer this, we constructed a version matrix spanning over 70 Docker releases and their associated runc versions, grouped into three eras: early (e.g., 17.03), mid-range (e.g., 20.10), and modern (e.g., 27.0.3). Each combination was tested using a consistent trick designed to detect CVE-2024-21626 (see Case Study 1).

Importantly, Houdini required no code or script modifications throughout this evaluation. Test logic remained unchanged. The orchestration layer, trick execution pipeline, and result reporting functioned identically across all tested Docker and runc versions. These findings validate Houdini's suitability as a version-agnostic security regression testing framework, capable of catching both intended fixes and unintended regressions across evolving container platforms.

This level of compatibility makes Houdini suitable not just for point-in-time security evaluation, but also for long-term regression tracking, archival vulnerability research, and continuous integration pipelines that must accommodate diverse and evolving container environments.

Table 5.6: Docker and runc Versions Validated with Houdini

Docker Version	**runc Version**	**Kernel Version**	**Validated**
17.03.0-ce	1.0.0-rc2	6.0	✓
17.03.1-ce	1.0.0-rc2	6.0	✓
17.03.2-ce	1.0.0-rc2	6.0	✓
17.04.0-ce	1.0.0-rc2	6.0	✓
17.05.0-ce	1.0.0-rc2	6.0	✓
17.06.0-ce	1.0.0-rc3	6.0	✓
17.06.1-ce	1.0.0-rc3	6.0	✓
17.06.2-ce	1.0.0-rc3	6.0	✓

Continued on next page

Table 5.6 – *continued from previous page*

Docker Version	**runc Version**	**Kernel Version**	**Validated**
17.09.0-ce	1.0.0-rc4+dev	6.0	✓
17.09.1-ce	1.0.0-rc4+dev	6.0	✓
17.12.0-ce	1.0.0-rc4+dev	6.0	✓
17.12.1-ce	1.0.0-rc4+dev	6.0	✓
18.03.0-ce	1.0.0-rc5	6.0	✓
18.03.1-ce	1.0.0-rc5	6.0	✓
18.06.0-ce	1.0.0-rc5+dev	6.0	✓
18.06.1-ce	1.0.0-rc5+dev	6.0	✓
18.06.2-ce	1.0.0-rc5+dev	6.0	✓
18.06.3-ce	1.0.0-rc5+dev	6.0	✓
18.09.0	1.0.0-rc5+dev	6.0	✓
18.09.1	1.0.0-rc6+dev	6.0	✓
18.09.2	1.0.0-rc6+dev	6.0	✓
18.09.3	1.0.0-rc6+dev	6.0	✓
18.09.4	1.0.0-rc6+dev	6.0	✓
18.09.5	1.0.0-rc6+dev	6.0	✓
18.09.6	1.0.0-rc6+dev	6.0	✓
18.09.7	1.0.0-rc8	6.0	✓
18.09.8	1.0.0-rc8	6.0	✓
18.09.9	1.0.0-rc8	6.0	✓
19.03.0	1.0.0-rc8	6.0	✓
19.03.1	1.0.0-rc8	6.0	✓
19.03.10	1.0.0-rc10	6.0	✓
19.03.11	1.0.0-rc10	6.0	✓
19.03.12	1.0.0-rc91	6.0	✓
19.03.13	1.0.0-rc10	6.0	✓
19.03.14	1.0.0-rc10	6.0	✓
19.03.15	1.0.0-rc10	6.0	✓
20.10.0	1.0.0-rc92	6.0	✓
20.10.10	1.0.2	6.0	✓

Continued on next page

Table 5.6 – *continued from previous page*

Docker Version	**runc Version**	**Kernel Version**	**Validated**
20.10.11	1.0.2	6.0	✓
20.10.12	1.0.2	6.0	✓
20.10.13	1.0.3	6.0	✓
20.10.14	1.0.3	6.0	✓
20.10.15	1.1.1	6.0	✓
20.10.16	1.1.1	6.0	✓
20.10.17	1.1.2	6.0	✓
20.10.18	1.1.4	6.0	✓
20.10.19	1.1.4	6.0	✓
20.10.20	1.1.4	6.0	✓
20.10.21	1.1.4	6.0	✓
20.10.22	1.1.4	6.0	✓
20.10.23	1.1.4	6.0	✓
20.10.24	1.1.5	6.0	✓
23.0.0	1.1.4	6.0	✓
23.0.1	1.1.4	6.0	✓
23.0.2	1.1.4	6.0	✓
23.0.3	1.1.5	6.0	✓
23.0.4	1.1.5	6.0	✓
23.0.5	1.1.5	6.0	✓
23.0.6	1.1.7	6.0	✓
24.0.0	1.1.7	6.0	✓
24.0.1	1.1.7	6.0	✓
24.0.2	1.1.7	6.0	✓
24.0.3	1.1.7	6.0	✓
24.0.4	1.1.7	6.0	✓
24.0.5	1.1.8	6.0	✓
24.0.6	1.1.9	6.0	✓
24.0.7	1.1.9	6.0	✓
24.0.8	1.1.11	6.0	✓

Continued on next page

Table 5.6 – *continued from previous page*

Docker Version	runc Version	Kernel Version	Validated
24.0.9	1.1.12	6.0	✓
25.0.0	1.1.11	6.0	✓
25.0.1	1.1.11	6.0	✓
25.0.2	1.1.12	6.0	✓
25.0.3	1.1.12	6.0	✓
25.0.4	1.1.12	6.0	✓
25.0.5	1.1.12	6.0	✓
26.0.0	1.1.12	6.0	✓
26.0.1	1.1.12	6.0	✓
26.0.2	1.1.12	6.0	✓
26.1.0	1.1.12	6.0	✓
26.1.1	1.1.12	6.0	✓
26.1.2	1.1.12	6.0	✓
26.1.3	1.1.12	6.0	✓
26.1.4	1.1.12	6.0	✓
27.0.1	1.1.13	6.0	✓
27.0.2	1.1.13	6.0	✓
27.0.3	1.1.13	6.0	✓

Conclusion

Houdini was tested with 73 different Docker versions and 20 versions of runc, all using the same Linux 6.0 kernel. Every version combination worked without needing to change any tricks or modify the framework. This shows that Houdini can handle a wide range of versions without breaking, even across more than ten years of Docker updates.

5.6 Performance Benchmarking

5.6.1 Understanding Time Metrics in Microbenchmarking

In performance benchmarking, especially in systems research and software profiling, several time-based metrics are commonly used to assess how a program or process utilizes computing resources. These include wall clock time, user time, system time, and CPU utilization. Each of these captures a different aspect of execution behavior and provides insight into how efficiently a task runs.

Wall clock time, also known as elapsed time or real time, is the total time taken from the start of a program to its completion. It includes all delays, such as waiting for I/O operations, context switches, or sleep intervals. Wall clock time is measured as the actual duration perceived by a user, typically using timestamps recorded before and after the process runs.

User time refers to the amount of CPU time spent running code in user space. This includes the program's own instructions and any library code it uses. It excludes time spent in the kernel or operating system. This metric is measured by the operating system and tracks how much CPU time was directly consumed by the application itself.

System time (or kernel time) accounts for the time spent by the CPU in the operating system kernel on behalf of the program. This includes handling system calls, managing I/O, allocating memory, and other interactions with hardware or privileged OS routines. Like user time, it is reported by the operating system's process accounting.

CPU utilization is the percentage of total available CPU time that the process actively used during its lifetime. It is computed as the ratio of the sum of user and system time to the wall clock time. High CPU utilization indicates that the process is CPU-bound, meaning it spends most of its time doing computations. Low CPU

utilization may suggest the process is I/O-bound or waiting on external events.

Together, these metrics offer a multifaceted view of a program's execution profile. They help developers identify bottlenecks, assess performance regressions, and understand whether inefficiencies stem from computation, system overhead, or external delays.

5.6.2 Microbenchmark

Making the Root Filesystem

To assess the performance of Houdini's build and setup pipeline, we independently microbenchmarked the `mkfs` stage, the process responsible for creating the root filesystem used by the VM. This stage includes both formatting the disk image and populating it with essential binaries, configuration files, and trick scripts drawn from Houdini's overlay directory.

Using the `/usr/bin/time` utility, we measured a typical execution time of `60.21 seconds`. The breakdown revealed a **user time** of 19.43 seconds and a **system time** of 45.97 seconds, resulting in an overall CPU utilization of approximately **63%**. These figures suggest that while the operation involves meaningful computation, a significant portion of the work is I/O-bound—largely due to file copying, directory creation, and image formatting tasks.

Table 5.7: Performance Metrics for `mkfs` Stage

Metric	**Value**
Wall-clock time	60.21 seconds
User time	19.43 seconds
System time	45.97 seconds
CPU utilization	63%

It is important to mention that the `mkfs` stage is a one-time setup cost per session. Once the filesystem image is created, it can be reused across multiple Houdini runs without needing to re-execute the build step. This reuse dramatically improves per-

formance in iterative workflows. In other words, By avoiding redundant execution of mkfs, subsequent Houdini runs skip over this 60-second cost entirely. In large batch runs or iterative experiments, this can yield cumulative time savings of several minutes or more. However, any modification to the overlay, such as adding or updating a trick requires rerunning `mkfs` to regenerate the root filesystem and incorporate the changes.

Booting the Virtual Machine

To assess the overhead of launching Houdini's virtual machine environment, we conducted a microbenchmark of the VM startup process. The VM is started using QEMU with a prebuilt Buildroot root filesystem and a custom Linux kernel. The system is configured to launch a shell and immediately power off after a fixed delay, ensuring consistency in measurement.

We measured the total wall-clock time required to boot the VM. The benchmark was performed using the `/usr/bin/time -v` utility, capturing both user and system time. The wall-clock time for this process was approximately 28.47 seconds. The user time recorded was 3.52 seconds, while the system time was 9.38 seconds. Overall, the job utilized roughly 45% of the CPU during execution.

Table 5.8: Performance Metrics for VM Boot

Metric	**Value**
Wall-clock time	28.47 seconds
User time	3.52 seconds
System time	9.38 seconds
CPU utilization	45%

These results indicate that the boot process introduces a moderate overhead in Houdini's workflow, primarily due to QEMU's emulation and the initialization of the Linux kernel and user space environment. Despite this, the startup time remains stable across multiple invocations, providing a consistent and reliable baseline for

empirical testing. This microbenchmark reinforces the practicality of using QEMU as a platform for repeatable, isolated container security experiments.

Building the Docker Image

To evaluate the performance overhead of Docker image creation within the Houdini pipeline, we conducted a microbenchmark of the `docker build` process. The benchmark was executed using the `/usr/bin/time -v` utility to capture detailed runtime metrics, including user time, system time, wall-clock time, and CPU utilization.

The wall-clock time to complete the initial build process was measured at approximately `1 minute and 13.04 seconds`. During this time, the process consumed `0.33 seconds` of user CPU time and `0.30 seconds` of system CPU time. The CPU utilization was reported at `0%`, indicating that the build process was largely I/O-bound, likely due to reading files, writing intermediate layers, and performing setup tasks for the container image.

Table 5.9: Performance Metrics for Docker Image Build (Cold Start)

Metric	**Value**
Wall-clock time	1 minute 13.04 seconds
User time	0.33 seconds
System time	0.30 seconds
CPU utilization	0%

It is important to note that this benchmark reflects the case where the Docker image did not previously exist. The initial build incurs higher latency due to downloading base layers, compiling dependencies, and writing the final image to disk. In subsequent builds where layers are cached and the image already exists locally, the build time is significantly reduced, and CPU utilization may increase due to reduced I/O waits.

These results highlight that while the initial Docker build may introduce measurable delay, especially in cold-start scenarios, this cost can be mitigated through

caching and reuse of previously built images. For workflows like Houdini, where consistency and speed are critical, avoiding redundant rebuilds is key to maintaining performance.

Running a Trick in Docker

To evaluate the runtime characteristics of executing a Houdini trick, we measured the performance of the command `docker run -it 504de41abf9b`. The benchmark revealed minimal resource usage, with `0.10 seconds` of user time and `0.07 seconds` of system time. The overall CPU utilization was measured at `9%`, suggesting that the trick involved limited computation and was mostly I/O-bound.

Table 5.10: Your Caption

Metric	**Value**
User time	0.10 seconds
System time	0.07 seconds
CPU utilization	9%

These results highlight Houdini's suitability for high-throughput, low-latency empirical testing, with minimal impact on system resources per trick execution.

Baseline Comparison Outside Houdini

To contextualize Houdini's efficiency, we measured a baseline workflow executed manually, without the automation Houdini provides. This baseline consisted of starting the VM, building the Docker image, and executing a security trick, which were all performed via shell scripts without caching or orchestration.

We used `/usr/bin/time -v` to capture system-level performance metrics. The manually executed baseline took approximately 55.70 seconds of wall-clock time, with a user time of 22.51 seconds and a system time of 18.12 seconds. The CPU utilization for this task was 73%, indicating moderate computational activity with some idle time.

Table 5.11: Performance Metrics for Manually Executed Baseline

Metric	Value
Wall-clock time	55.70 seconds
User time	22.51 seconds
System time	18.12 seconds
CPU utilization	73%

Compared to Houdini's streamlined execution, which achieves the same test in a fraction of the time through reuse of cached artifacts and fewer manual dependencies, this demonstrates that Houdini significantly reduces runtime overhead.

5.7 Batch Trick Execution

To evaluate Houdini's performance under batch workloads, we executed a session consisting of 32 distinct tricks across a single version of Docker and kernel. The goal was to measure the system's ability to handle high-throughput empirical testing within a single execution context. This benchmark focused on key metrics such as total runtime, CPU usage, and failure rates.

The benchmark was conducted using the `/usr/bin/time -v` utility to capture detailed performance statistics. The total wall-clock time for executing all 32 tricks was approximately 1 minute and 6.80 seconds. During this period, the system reported a user time of 2.75 seconds and a system time of 0.48 seconds, with an overall CPU utilization of just 4%. These results indicate that while the batch execution is time-bound primarily by I/O and process orchestration overhead, the actual CPU demand remains minimal, suggesting scalability across larger test sets.

5.8 Scalability via Bulk Execution

To evaluate scalability, we constructed a batch consisting of 32 distinct trick-configuration pairs. Each trick was executed in a fresh QEMU VM using a consistent Buildroot

Table 5.12: Performance Metrics for Batch Execution of 32 Tricks

Metric	Value
Total tricks executed	32
Wall-clock time	66.80 seconds
User time	2.75 seconds
System time	0.48 seconds
CPU utilization	4%

image, Docker version, and Linux kernel (v6.0). The test matrix covered different combinations of runtime flags and container settings to reflect real-world usage during regression testing.

Listing 5.13: Trick Configuration Parameters

```
pid_mode = [None, "--pid=host"]
cap_kill = [False, True]
cap_sys_ptrace = [False, True]
privileged_opts = [False, True]
```

Houdini's batch system accepted this matrix as a single declarative input and automatically expanded it into 32 independent tasks. Each task was queued and executed without manual intervention. The entire batch completed successfully in a single uninterrupted session, confirming the framework's ability to handle moderate-scale workloads without user supervision.

Throughout the bulk execution of 32 trick-configuration pairs, Houdini successfully generated a separate log file for each individual test. Each log captured the complete stdout output as well as the full Docker command used to launch the container (including all runtime flags). For example:

```
docker run --rm --cap-add=CAP_KILL --pid=host ...
Trick passed
```

All logs were uniquely named based on the trick and configuration, stored in a structured directory hierarchy corresponding to the test batch. Manual inspection

confirmed that every log was complete, correctly formatted, and attributed to the correct trick. No logs were missing, truncated, or overwritten.

This logging mechanism supports both debugging and reproducibility, as each file serves as a self-contained record of the execution. Moreover, the trick output in every log matched the expected behavior for its configuration, affirming the functional correctness of the batch.

While this evaluation involved 32 trick-configuration pairs, Houdini's architecture does not impose a hard-coded upper limit on batch size. The system is designed to accept arbitrarily large test matrices and automatically expand and schedule each test. In principle, Houdini can scale to hundreds or even thousands of tasks.

This evaluation confirms that Houdini meets its scalability goal. The system can reliably orchestrate dozens of isolated tests in a single batch, automatically handling setup, teardown, and result collection. As the matrix of container configurations continues to grow with new software versions and security flags, this scalability ensures that Houdini remains suitable for long-term and large-scale security regression testing.

Chapter 6

Case Studies

6.1 Evaluating the accuracy of CVE-2024-21626

CVE-2024-21626 [5] is a container escape vulnerability in runc, the low-level container runtime used by Docker and other container engines. It allows a malicious container to escape its isolation and execute arbitrary commands on the host system.

The root cause of this vulnerability lies in how runc handles cgroups for containers, which are used to manage system resources like CPU and memory. During container creation, runc opens a file descriptor to the host's `/sys/fs/cgroup` directory to configure resource limits. This descriptor remains open throughout the container's lifecycle and is exposed inside the container via `/proc/self/fd/`, which lists all open file descriptors of the running process. The vulnerability arises when an attacker sets the working directory in the Dockerfile to point to one of these exposed descriptors, such as `/proc/self/fd/N`, where `N` refers to the specific file descriptor inherited from runc that points to the host's cgroup hierarchy. This causes the container's current working directory to resolve to a privileged host path. As a result, all subsequent relative file operations in the container are interpreted from the host's cgroup filesystem rather than the container's own root, allowing unintended access or modification of host resources.

This behavior is enabled by Linux's treatment of `/proc/self/fd/N` as dynamic

symlinks to open file descriptors. When dereferenced, these symlinks resolve to the original host-side path, regardless of container filesystem restrictions. Because runc and Docker do not restrict access to inherited file descriptors or sanitize the working directory path during initialization, attackers can exploit this oversight to escape isolation. By simply changing the container's working directory to a leaked file descriptor, a malicious container can directly interact with host-managed control groups. To test the exploitability of CVE-2024-21626, we designed a Houdini trick that executes the following steps on Docker and runc versions listed in Table 6.1:

1. Set the working directory to `/proc/self/fd/<fd>`, pointing to a controlled file descriptor.

2. Attempt to break out of the container by leveraging the misconfigured working directory.

3. Validate whether commands can be executed on the host system.

The CVE for this vulnerability indicates that the affected runc versions are $\geq$ `1.0.0-rc93`, $\leq$ `1.1.11`. This suggests the vulnerability was introduced in runc version rc93 and fixed in version 1.1.12. However, our tests conducted with Houdini revealed a significant discrepancy with this assertion. Our testing showed that earlier runc versions, such as `1.0.0-rc1`, `1.0.0-rc2`, `1.0.0-rc3`, and `1.0.0-rc4` also exhibited the same vulnerability. This discovery expands the scope of the CVE by revealing that the issue existed well before version `rc93`.

This discrepancy highlights a recurring issue in vulnerability disclosure processes: CVEs often rely on assumptions from static code analysis or limited testing. Without rigorous empirical validation across historical versions, critical edge cases can be missed. In the case of CVE-2024-21626, this oversight could have allowed vulnerable deployments relying on older runc versions to remain unpatched under the false belief that they were unaffected.

Table 6.1: runc versions against CVE-2024-21626

Docker Version	Runtime Version	Leaked FD	Exploit Success
-	runc 0.0.9	-	✗
1.12.0	runc 1.0.0-rc1	/proc/self/fd/4	✓
17.03.0-ce	runc 1.0.0-rc2	/proc/self/fd/4	✓
17.06.1-ce	runc 1.0.0-rc3	/proc/self/fd/5	✓
17.09.0-ce	runc 1.0.0-rc4	/proc/self/fd/5	✓
-	runc 1.0.0-rc8	-	✗
-	runc 1.0.0-rc10	-	✗
-	runc 1.0.0-rc91	-	✗
20.10.17	runc 1.1.2	/proc/self/fd/9	✓
23.0.1	runc 1.1.4	/proc/self/fd/8	✓
23.0.5	runc 1.1.5	/proc/self/fd/8	✓
24.0.2	runc 1.1.7	/proc/self/fd/9	✓
24.0.5	runc 1.1.8	/proc/self/fd/9	✓
24.0.7	runc 1.1.9	/proc/self/fd/9	✓
24.0.0	runc 1.1.11	/proc/self/fd/9	✓
-	runc 1.1.12	-	✗
-	runc 1.1.13	-	✗
-	runc 1.1.14	-	✗

6.2 Evaluating Host Process Manipulation

In this evaluation, we examine whether a Docker container running as a non-privileged user with `UID 1000` can interact with and manipulate host processes that are also owned by `UID 1000`. Under a default Docker configuration, the PID namespace provides process isolation by restricting a container's visibility and access to processes outside its own namespace. However, when the container is launched with the `-pid=host` flag, it shares the host's PID namespace, allowing it to enumerate and interact with host processes. This configuration effectively bypasses one of the core isolation mechanisms provided by containers. To assess the security implications of this namespace sharing, we investigate whether process-level interactions, such as sending signals or attaching debuggers are permitted between the container and host processes when UID ownership is aligned. The testing environment for this case study is in Table 6.2.

Table 6.2: Case Study 2 Testing Environment

Component	Specification
Docker Version	27.0.3
Container Runtime	`runc` 1.1.13

Case study 2's results are summarized in Table ??, where all pairs of flags were tested using Houdini, and where the cells show what functionality was possible when launching containers with the pair of flags intersecting the rows and columns of Table ??.

Our objective here is to investigate how various security flags interact with each other, and to determine the minimum set of privileges required for a container to manipulate host processes. Specifically, we looked at the following flags:

- `-pid=host`
- `CAP_KILL`
- `CAP_SYS_PTRACE`
- `-privileged`
- `apparmor=docker-default`

The evaluation began by assessing whether a container launched with `-pid=host` could list host processes using `ps aux`. As expected, the container displayed all processes running on the host, confirming that sharing the host PID namespace fully exposes the host's process table to the container.

Following this, we tested whether a container process could send a `SIGKILL` signal to a host process owned by `UID 1000`. The container was launched with `-pid=host`, allowing it to see and interact with host processes. Despite not being granted `CAP_KILL`, the signal was successfully delivered. This is because by default, both processes share the same UID and PID namespace, and under standard Linux permission checks, a process can send signals to another process it owns by UID

without requiring special capabilities. However, if the target host process is owned by a different UID (another non-root user), `CAP_KILL` is required for the signal to be delivered. That capability must also be effective in the user namespace of the target process. Similarly, signaling host processes owned by `UID 0` remains restricted even with `CAP_KILL`, unless the container process itself is also running as root.

Combining `-pid=host` with `CAP_SYS_PTRACE` enables nearly the same level of interaction with host processes as granting `-privileged` mode. Specifically, it allows a containerized process to list host processes, send signals, and attach a debugger to them, all key capabilities typically associated with privileged containers. However, unlike `-privileged`, this more targeted configuration avoids giving the container unnecessary capabilities such as access to all of the host's devices. These results demonstrate that it is unnecessary to over-privilege containers when finer-grained controls can achieve the same functional outcomes with a significantly better security posture.

When the container is confined by AppArmor's `docker-default` profile, its ability to interact with host processes becomes significantly restricted. Specifically, `docker-default` explicitly denies system calls such as `signal` and `ptrace`, which are required for sending signals and attaching debuggers to other processes. As a result, even if the container shares the PID namespace with the host and possesses the necessary capabilities, these operations will fail.

However, including the `-privileged` flag when starting the container overrides AppArmor confinement by disabling the `docker-default` profile entirely. This lifts all AppArmor-imposed restrictions, allowing the container full access to the previously restricted system calls. As a result, operations that were previously blocked, such as sending signals or attaching debuggers to host processes are now permitted, even if AppArmor would normally deny them.

It is also important to note that granting capabilities such as `CAP_KILL` and

`CAP_SYS_PTRACE` alone is not sufficient for interacting with host processes unless the container shares the host's PID namespace. Without the `-pid=host` flag, the container operates in its own isolated PID namespace, meaning it cannot see or reference host PIDs at all. As a result, any attempt to send signals or attach to host processes will fail.

We note that the explanation above was derived after substantial amount of testing and analysis. The Docker documentation alone does not clarify this behavior. Instead, users are expected to understand both the Docker tooling and the underlying Linux kernel security model to correctly select these flags.

When the container shares the same user namespace as the host (or has aligned UID mappings), and the container process runs under the same UID as the host processes, it can bypass security restrictions that would normally prevent such interactions. In this case, the container may be able to perform operations like sending signals or attaching debuggers to host processes, even without elevated capabilities like `CAP_KILL` or `CAP_SYS_PTRACE`. These capabilities, which are typically required for such actions, are not necessary when the container and host share UID mappings, as the container process is considered the same user as the host processes it targets.

This case study highlights a core strength of Houdini, which is its ability to systematically explore and identify the least privilege for a given operation. By allowing users to declaratively modify container configurations, Houdini makes it easy to test a wide range of privilege combinations in a repeatable and automated way. Instead of relying on trial and error or defaulting to the overly permissive `-privileged` mode, users can incrementally grant permissions and observe the effect of each change. Houdini thus enables security practitioners, developers, and researchers to precisely determine the minimum set of permissions required for a given trick to succeed. As a result, Houdini not only serves as a testing framework for container escapes and misconfigurations, but also as a practical tool for privilege minimization and hardening

container deployments.

This case study highlights the complexity of security policies in containerized environments, highlighting that the runtime behavior of security configurations is not always straightforward or predictable. While administrators may configure a container with `-pid=host` to enable the container to monitor host processes, there is an assumption that certain operations, such as sending signals or attaching debuggers remain restricted. However, these assumptions may not hold at runtime due to the interplay of various factors such as user namespaces and UID mappings.

Table 6.3: Ability for Containers to Interact with Host Processes Across Yama Settings (Ubuntu Kernel 6.8.12)

Security Configuration	Yama Scope	-pid=host	CAP_KILL	CAP_SYS_PTRACE	-privileged	apparmor=docker-default	-pid=host + apparmor=docker-default
-pid=host	0	◇ △	◇ △	◇ △ ⋆	◇ △ ⋆	◇	◇
	1	◇ △	◇ △	◇ △ ⋆	◇ △ ⋆	◇	◇
	2	◇ △	◇ △	◇ △	◇ △	◇	◇
	3	◇ △	◇ △	◇ △	◇ △	◇	◇
CAP_KILL	0	◇ △					◇
	1	◇ △					◇
	2	◇ △					◇
	3	◇ △					◇
CAP_SYS_PTRACE	0	◇ △ ⋆					◇
	1	◇ △ ⋆					◇
	2	◇ △					◇
	3	◇ △					◇
-privileged	0	◇ △ ⋆					◇ △ ⋆
	1	◇ △ ⋆					◇ △ ⋆
	2	◇ △					◇ △
	3	◇ △					◇ △
apparmor=docker-default	0	◇					◇
	1	◇					◇
	2	◇					◇
	3	◇					◇
-pid=host + apparmor=docker-default	0	◇	◇	◇	◇ △ ⋆	◇	◇
	1	◇	◇	◇	◇ △ ⋆	◇	◇
	2	◇	◇	◇	◇ △	◇	◇
	3	◇	◇	◇	◇ △	◇	◇

Table 6.4: *
Tests run as a non-root user (UID=1000) against a host process owned by UID 1000.
◇ List Host Process △ Send Signal to Host Process ⋆ Attach Debugger to Host Process

6.3 Evaluating Docker Privileged Port Forwarding Restrictions

In this case study, we evaluate Docker's network isolation mechanisms to determine how well they enforce privileged port binding restrictions. Starting from Docker version 20.03, unprivileged ports start at 0 instead of 1024, effectively removing the

need for root to bind to privileged ports. Table 6.5 presents the results of this change.

Table 6.5: Ability for a container to bind to privileged ports

Docker Version	NET_BIND_SERVICE + root	NET_BIND_SERVICE + non-root user	root	non-root user	Default Contai
19.03.9	✓	✗	✗	✗	✗
19.03.0	✓	✗	✗	✗	✗
20.10.0	✓	✗	✗	✗	✗
20.10.3	✓	✗	✗	✗	✗
23.0.0	✓	✓	✓	✗	✗
25.0.0	✓	✓	✓	✗	✗
27.0.1	✓	✓	✓	✗	✗

Table 6.6: *

Note: A non-root user implies a user account that does not have administrative privileges, (not root-less Docker).

This case study highlights how discrepancies between Docker versions in handling privileged port binding can create gaps between administrators' intended confinement policies and actual enforcement. A change in Docker's handling of binding to privileged ports, such as the removal of the NET_BIND_SERVICE requirement starting from Docker 20.03, may allow containers to bypass restrictions even when policies are set to prevent it.

For example, administrators might configure a container to be restricted from binding to privileged ports, but if Docker is running a version where the NET_BIND_SERVICE capability is no longer required for root containers (like in Docker 20.03 and beyond), the container could still bind to these ports despite the policy specifying otherwise. This creates a mismatch between the intended policy (restricting privileged port binding) and the actual enforcement (where containers can still bind to those ports in certain Docker versions).

Once again, Houdini helps identify these gaps by systematically testing container configurations across different Docker versions. By running different Docker versions and security policies, Houdini verifies whether containers are correctly enforcing the intended restrictions on privileged port binding. For instance, Houdini could test whether containers configured to prevent binding to privileged ports are still able to bind to those ports in Docker versions 20.03 or later, where the security model has changed. If a container running with Docker 20.03 or newer can bind to privileged

ports despite the configuration specifying otherwise, Houdini will detect this discrepancy and highlight the gap between the intended policy and actual enforcement.

This set of experiments evaluates how Docker's privilege model for binding to privileged ports (i.e., ports below 1024) has evolved across versions. Traditionally, binding to privileged ports required root access or the NET_BIND_SERVICE capability. However, newer versions of Docker introduced support for binding to these ports even in rootless containers, provided the necessary capability is explicitly granted.

By testing various combinations of user privileges (root vs. rootless) and capability settings across multiple Docker versions, this case study reveals which configurations are sufficient and which are excessive for achieving the goal of binding to a low-numbered port. It demonstrates that in modern Docker versions (23.0.0+), it is no longer necessary to run as root or grant all privileges to perform this operation. Instead, a least-privileged approach using a rootless container with NET_BIND_SERVICE is sufficient.

This case study shows that Houdini can empower Docker users to avoid unnecessary elevation of privileges, particularly avoiding full root access or the use of the –privileged flag, both of which expose containers to a broader attack surface. Houdini enables this kind of analysis by systematically testing Docker configurations and identifying the minimum set of privileges required for a given operation, helping developers and administrators adopt safer, more secure defaults.

This case study also demonstrates a clear example of configuration drift arising not from explicit changes made by administrators, but from underlying changes in the Docker runtime itself. As Docker evolves, fundamental shifts in its privilege model, such as the redefinition of which ports are considered privileged and the removal of the NET_BIND_SERVICE requirement for certain operations can silently undermine previously established security policies. Even if an organization maintains the same container configuration and security policies, upgrading Docker to a newer version

can inadvertently expand container capabilities, allowing behaviors that were once restricted. This version-induced drift can lead to a false sense of security, where administrators believe containers are still confined by earlier policy expectations, when in fact those policies are no longer fully enforced.

6.4 Evaluating ptrace Enforcement Across Linux Distributions

To assess the effectiveness of Linux kernel mechanisms in enforcing process isolation within containerized environments, we conducted a series of controlled experiments targeting the ptrace system call under various configurations. These experiments were designed to evaluate how different kernel versions, Yama security module settings, and process capabilities affect the ability of a containerized process to trace other processes, both inside and outside its container boundary.

To conduct this study, we used Houdini to run a standardized trick that attempts to use ptrace(PTRACE_ATTACH) inside a Docker container. The experiments were run across 10 major Linux distributions, covering kernel versions ranging from 2.6.32 to 6.8. Each distribution was tested using its default Yama ptrace_scope setting, as found in /proc/sys/kernel/yama/ptrace_scope. For each configuration, we tested the behavior both with and without CAP_SYS_PTRACE, isolating the influence of this capability on tracing permissions.

Each test followed the same methodology. A container was launched with a known configuration, and a background process was started within it. The container then attempted to attach to this process using ptrace. A successful or failed attempt was recorded based on the return code of the ptrace system call and the associated error message.

The results, summarized in Table 6.7, reveal strong correlations between the de-

fault Yama scope and ptrace enforcement. Distributions using Yama scope 1, such as Ubuntu 22.04, Ubuntu 24.04, Debian 12, and Arch Linux, restricted ptrace to processes with the same UID, unless CAP_SYS_PTRACE was present. In these systems, removing the capability consistently caused the ptrace attempt to fail, indicating correct enforcement. In contrast, distributions that default to Yama scope 0, such as Debian 8, Alpine 3.19, and CentOS 6, allowed tracing even without CAP_SYS_PTRACE, which poses a potential security risk. Fedora 40, which defaults to Yama scope 2, exhibited the strictest behavior: ptrace failed in all test cases, even when the container was granted CAP_SYS_PTRACE, demonstrating a highly restrictive enforcement model. These results also confirm that the kernel version alone is not sufficient to predict ptrace behavior; the default Yama configuration plays a critical role.

Overall, this case study highlights the lack of uniformity in ptrace enforcement across Linux distributions.

Table 6.7: ptrace Behavior Across Distributions with Default Yama Scopes

Distribution	**Kernel Version**	**Yama Scope (Default)**	**CAP_SYS_PTRACE**	**Result**
Debian 12	6.1 (Bookworm)	1	Yes	Success
Debian 12	6.1 (Bookworm)	1	No	Fail
Debian 8	3.16 (Jessie)	0	Yes	Success
Debian 8	3.16 (Jessie)	0	No	Success
Fedora 40	6.7	2	Yes	Fail
Fedora 40	6.7	2	No	Fail
Ubuntu 24.04	6.8.12	1	Yes	Success
Ubuntu 24.04	6.8.12	1	No	Fail
Ubuntu 22.04	5.15 (LTS)	1	Yes	Success
Ubuntu 22.04	5.15 (LTS)	1	No	Fail
Ubuntu 16.04	4.4 (LTS)	0	Yes	Success
Ubuntu 16.04	4.4 (LTS)	0	No	Success
Alpine 3.19	6.1	0	Yes	Success
Alpine 3.19	6.1	0	No	Success
Arch Linux	6.8 (Rolling)	1	Yes	Success
Arch Linux	6.8 (Rolling)	1	No	Fail
CentOS 7	3.10	1	Yes	Success
CentOS 7	3.10	1	No	Fail
CentOS 6	2.6.32	0	Yes	Success
CentOS 6	2.6.32	0	No	Success

Chapter 7

Discussion

The evaluation section demonstrated Houdini's ability to systematically uncover gaps between intended and actual security guarantees in containerized environments. Through a series of case studies, we showed how Houdini can reveal vulnerabilities that persist across multiple runtime versions (as with CVE-2024-21626), analyze the impact of privilege and namespace flags on host interaction, and detect configuration drift resulting from runtime or policy changes. Collectively, these results highlight the value of empirical, reproducible testing for strengthening container confinement.

It is important to emphasize that Houdini's applications extend beyond the specific scenarios examined in this paper. The modular, extensible design enables users to create and share new tricks for testing a wide range of security properties, misconfigurations, or policy violations. As a general-purpose framework, Houdini is poised to support the broader research and practitioner community in addressing evolving security challenges in containerized systems, including those not explicitly covered here.

Houdini's design not only addresses critical security challenges in container confinement but also lends itself well to continuous security testing in modern software development environments. Because it is both safe and modular, Houdini can be

seamlessly integrated into automated systems, such as CI/CD pipelines, to provide ongoing validation of container security.

Regression testing is a widely adopted practice in software engineering to ensure that updates or changes to code do not introduce unintended failures or security regressions. In the context of container security, integrating Houdini into CI/CD pipelines enables organizations to automatically trigger a comprehensive suite of confinement tests whenever any part of the container stack changes, such as kernel upgrades, Docker releases, or policy updates. This continuous validation ensures that every new deployment or system update is accompanied by an up-to-date security assessment. By systematically rerunning empirical security tests across different software versions and configurations, Houdini helps detect configuration drift and regression in security guarantees that might otherwise go unnoticed, maintaining the effectiveness of container isolation mechanisms as platforms evolve.

Finally, to foster further research and encourage community adoption, we have made the source code for Houdini publicly available[1], along with a base set of initial tricks. We hope this will enable others to extend Houdini's capabilities, contribute new test cases, and collectively advance the state of container security benchmarking.

[1] `https://github.com/CU-Security-Labs/houdini`

Bibliography

[1] William Findlay, David Barrera, and Anil Somayaji. Bpfcontain: Fixing the soft underbelly of container security. *arXiv preprint arXiv:2102.06972*, 2021.

[2] CrowdStrike. VENOM: Virtualized Environment Neglected Operations Manipulation. Online, 2015. `https://www.crowdstrike.com/blog/cve-2015-3456-venom/`.

[3] Paul Kocher, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz, and Yuval Yarom. Spectre attacks: Exploiting speculative execution. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, pages 1–19, 2019.

[4] Michael Schwarz, Daniel Gruss, Moritz Lipp, and Stefan Mangard. ZombieLoad Attack. Online, 2019. `https://zombieloadattack.com/`.

[5] Open Containers Maintainers. runc: File descriptor leak leading to container breakout (CVE-2024-21626). `https://github.com/opencontainers/runc/security/advisories/GHSA-xr7r-f8xq-vfvv`, 2024. Accessed: 2024-04-28.